

ANÁLISIS DE SISTEMAS - UNIDADES

UNIDAD N°1: EL MODELADO DE PROCESO DE NEGOCIO

- Los sistemas de información y los procesos de negocio.
- Concepto de Modelo. Fundamentos de su uso.
- Conceptos de: proceso, proceso de negocio, actividades del proceso, reglas de negocio, modelado de procesos de negocio, importancia, técnicas, herramientas.
- Identificación de procesos. Descripción y modelado gráfico de procesos.
- Introducción a BPMN, notación.

UNIDAD N°2: SISTEMAS DE INFORMACIÓN Y PROCESOS DE DESARROLLO DE SISTEMAS DE INFORMACIÓN Y SISTEMAS DE SOFTWARE

- Definición de los requerimientos de un sistema de información. Concepto de requerimientos en el contexto de los sistemas de información.
- Conceptos de: ingeniería de software, método, técnicas, herramientas, proceso, proyecto.
- El proceso de desarrollo, metodologías, herramientas, roles y artefactos.
- Modelos de proceso: concepto. Importancia. Distintos modelos de proceso. Conceptos de calidad en el desarrollo de software. Estudio de prefactibilidad.
- Presentación del Proceso Unificado de Desarrollo.
- Metodologías Ágiles. Surgimiento. El manifiesto ágil. Problemáticas y evolución de las metodologías tradicionales y ágiles.

UNIDAD N°3: EL PARADIGMA DE LA ORIENTACIÓN A OBJETOS

- Concepto de paradigma. Paradigma Orientado a Objetos. Elementos del paradigma: Objeto, naturaleza, estado, comportamiento, clase, vistas.
- Elementos del modelo de objetos, relaciones entre objetos, relaciones entre clases.
- Modelo de objetos del dominio del problema. Patrones para el modelado de dominio.
- El Lenguaje Unificado de Modelado (UML): surgimiento, elementos, estructura. Aspectos generales del lenguaje.
- Herramientas para el modelado de objetos. Diagrama de clases.

UNIDAD N°4: INGENIERÍA DE REQUERIMIENTOS

- Requerimientos: concepto, categorías, tipos.
- Ingeniería de requerimientos. Fundamentos, propósito.
- Procesos de la ingeniería de requerimientos: elicitación, especificación, validación.

UNIDAD N°5: EL PROCESO DE ELICITACIÓN DE REQUERIMIENTOS

- La elicitación de requerimientos: concepto, importancia.
- Fuentes de información.
- Técnicas de recopilación de información: entrevista, cuestionario, análisis de documentación, observación, torbellino de ideas.

UNIDAD N°6: EL PROCESO DE ESPECIFICACIÓN DE REQUERIMIENTOS

- La especificación de requerimientos: concepto, importancia.
- Captura de requerimientos con casos de uso.
- Modelo de casos de uso: diagrama de casos de uso. Descripciones de casos de uso. Patrones de casos de uso.
- Modelo de objetos del dominio en la especificación de requerimientos. Consistencia e integración.
- Prototipo de interfaz.
- Diagrama de máquina de estados.
- Consistencia e integración de modelos de especificación de requerimientos.
- El documento de especificación de requerimientos de software.

UNIDAD N°7: EL PROCESO DE VALIDACIÓN DE REQUERIMIENTOS

- La validación de requerimientos: concepto, importancia, tipos.
- Técnicas, prototipos.

UNIDAD N°8: PROCESO UNIFICADO DE DESARROLLO. FLUJOS DE TRABAJO DE REQUISITOS Y ANÁLISIS.

- El proceso unificado de desarrollo: concepto, fases, flujos de trabajo.
- El flujo de trabajo de requisitos: concepto, importancia.
- Artefactos, trabajadores y actividades del flujo de trabajo de requisitos.
- El flujo de trabajo de análisis: concepto, importancia.
- Artefactos, trabajadores y actividades del flujo de trabajo de análisis.

Unidad 1: Modelado de Procesos de Negocio

Gestión de procesos	2
Procesos de negocio	3
¿Qué es un proceso?	3
Visión de procesos.....	3
Gestión de procesos.....	3
Reglas de negocio.....	3
Clientes del proceso	4
Dueño del proceso	4
Tipos de procesos.....	4
Estratégicos	4
Esenciales	4
De soporte o apoyo	4
Modelado visual de los Procesos	5
Mapa de procesos	5
Mapa del ámbito	5
Flujograma.....	6
Tareas de la actividad.....	6
Análisis de la ficha de proceso de negocio.....	7
Modelado con BPMN	8
1- Objetos de flujo	8
2- Objetos de conexión	9
3- Artefactos.....	9
4- Swimlanes (canales o pools)	10
5- Objetos/almacén de datos.....	10
Niveles de granularidad.....	10

Unidad 1: Modelado de Procesos de Negocio

Gestión de procesos

Técnica de gestión que ayuda a los dueños de los procesos a identificar, diseñar, formalizar, controlar, mejorar y hacer más productivos los procesos de la organización para lograr la confianza del cliente.

GESTIÓN FUNCIONAL	GESTIÓN POR PROCESOS
Se basa en organización por áreas. La autoridad se basa en Jefes de Área. Existe una jerarquía de poder.	Se basa en organización por procesos. La autoridad se basa en Dueños de Proceso Se le da más valor al cliente.
Se dirige por vigilancia y control, y el trabajo está condicionado por necesidades.	Se dirige con apoyo, supervisión, y existe la autonomía. Tiene una visión totalitaria, es decir que analiza toda la situación, de principio a fin.
<u>Principio de eficacia:</u> se enfoca en hacer más y en cómo se hacen los procesos.	<u>Principio de eficacia:</u> enfoque en hacer lo correcto eficientemente y en el “para quién lo hacemos”

Procesos de negocio

¿Qué es un proceso?

Conjunto de actividades, interacciones y recursos con una finalidad en común: transformar las entradas en salidas y que éstas agreguen valor para el cliente.

Visión de procesos

Permite comprender la complejidad en las interacciones entre personas y acciones distantes en el tiempo y el espacio.

Ofrece una vista horizontal y da una respuesta a un ciclo completo.

Se tiene en cuenta los límites del proceso, es decir **desde** que se produce contacto con el cliente **hasta** haber producido una serie de transformaciones de entradas y salidas, logrando un resultado llamado **producto**.

Gestión de procesos

Se busca tener una nueva visión que favorezca y facilite al negocio a mantenerse en el negocio competitivo.

Con esto, se consigue cambiar la forma de actuar de los dirigentes de las organizaciones y lograr mejores resultados en los procesos, de forma continua, eficaz y metodológicamente.

Además, se **agrega valor al cliente**, le dan un producto de valor.

Se trabaja en tres apartados:

Describir: identificar procesos, quienes intervienen y cómo funcionan.

Mejorar: optimizar un proceso a través de algún cambio.

Rediseñar: replantear la forma en que se hace, y formar una nueva propuesta.

Reglas de negocio

Sentencia que define cómo el negocio debería ejecutarse y/o como los recursos se pueden estructurar y relacionarse con otros.

Describen los lineamientos, políticas y comportamientos del modo de trabajar del negocio.

Clientes del proceso

Ya teniendo en cuenta que los procesos existen para darle valor al cliente, se pueden diferenciar entonces varios tipos de clientes:

Internos: otro/s procesos de la organización que requieren de otro/s.

Externos: son los relacionados directamente con la misión de la organización.

Stakeholders: grupo de interés que, si bien no están relacionados de forma directa, pueden llegar a crear una restricción. Ej.: los accionistas.

Dueño del proceso

Es aquel que está dentro de la organización y tiene un rol ejecutivo.

Es el que **conoce básicamente todo acerca de la organización**.

Tiene **autoridad total** para modificar los procesos.

Su misión es siempre buscar **aumentar la satisfacción** del cliente.

Tipos de procesos

Estratégicos

Establecen estrategias, normativas, instructivos, en la organización.

Dan un marco o visión general que necesita hacer la organización para funcionar.

Esenciales

También se denominan procesos del negocio.

Son aquellos que **atienden a la misión de la organización**, y los que **se relacionan con el cliente**.

Pueden ser de 1 a 3 en empresas pequeñas, y de 5 a 6 en más grandes.

De soporte o apoyo

Ejecutan un conjunto de acciones que son **complementarias y necesarias** para que los procesos esenciales se puedan desarrollar correctamente.

Pueden ser fácilmente 20 en empresas pequeñas, y más de 200 en más grandes.

Modelado visual de los Procesos

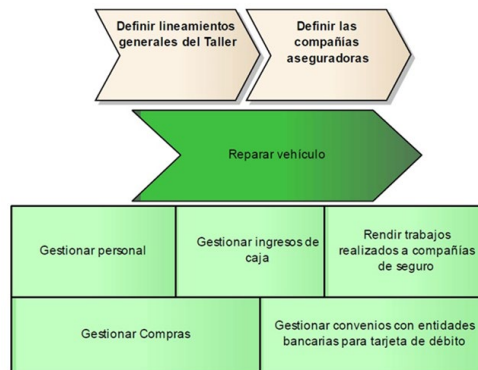
Las clasificaciones anteriores se pueden representar mediante diagramas visuales.

Se modelan procesos de negocios para, por ejemplo, conocer **cómo funcionan los mismos**. A través de esto, se puede determinar mejor el **sistema de información más adecuado** que soporte al negocio.

Se pueden utilizar cuatro modelos:

Mapa de procesos

Diagrama que identifica los procesos según las tres clasificaciones anteriores.



Mapa del ámbito

Detalla **una parte** de un proceso (segmentación de proceso).

Facilita la intervención del proceso, permite describir, mejorar o rediseñar.

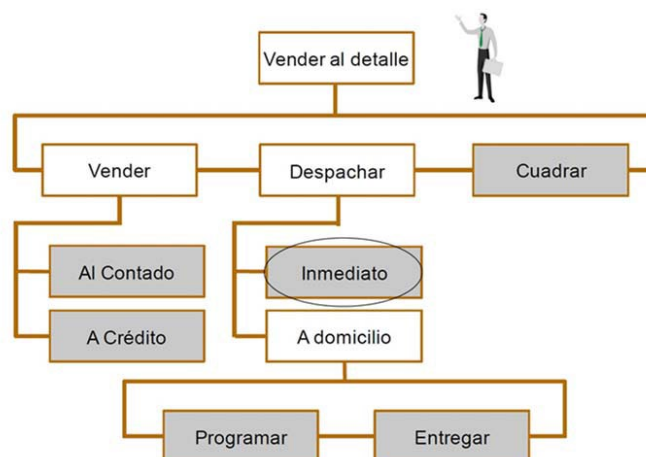


Figura 1-4. Mapa del ámbito Vender al detalle del proceso Comercializar

Flujograma

Muestra un **vistazo general** de un proceso que puede ser complejo.

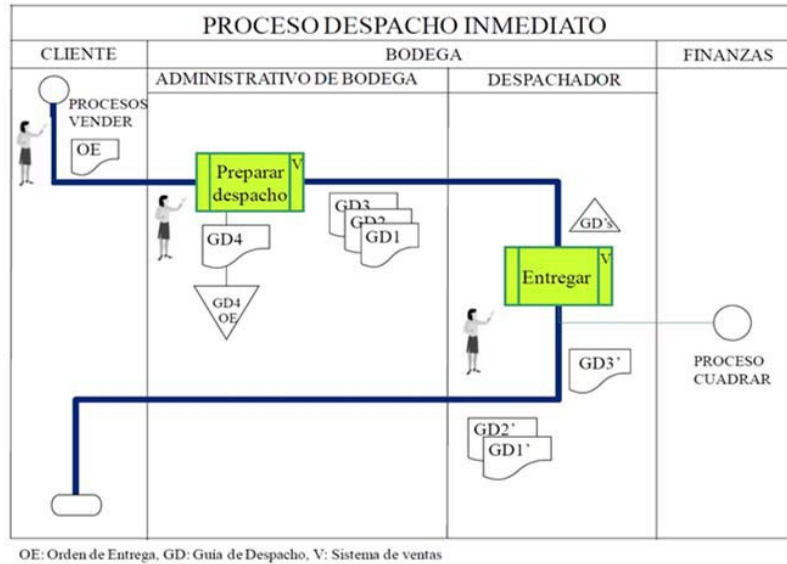


Figura 1-5. Flujograma de información del proceso operativo Despacho Inmediato

Tareas de la actividad

Muestra en **forma de lista, tareas** de una actividad como, por ejemplo, entregar.

LISTA DE TAREAS DE LA ACTIVIDAD: ENTREGAR	
TAREA	DESCRIPCIÓN DE LA TAREA
Tomar GD	Desde la carpeta transitoria forma FIFO, los 3 ejemplares de la GD.
Buscar producto	En la bodega según ubicación indicada en la GD
Registrar	En la ficha de producto ubicada en el estante destinado al producto (registro)
Rebajar stock	En el mesón de despacho, usa el lector de código de barras
Verificar producto	Junto con el cliente a quien pide firmar los 3 ejemplares de la GD (punto de control)
Entregar al cliente	Le entrega los ejemplares 1 y 2 de la GD y el producto
Enviar a finanzas	A través del estafeta una vez al final de día

Figura 1-6. Lista de tareas de una actividad

Análisis de la ficha de proceso de negocio

Proceso de Negocio: < Nombre Proceso de Negocio>		
Objetivo	<Objetivo del proceso de negocio>	
Cliente del Proceso	<Nombre del cliente identificado del proceso de negocio>	
Producto del Proceso	<Solo la salida principal, resultado de cumplir con el objetivo del proceso>	
Proveedores del proceso e insumos que brinda cada uno	Proveedor	Insumo
	<Nombre de otro proceso proveedor, entidades externas, personas o roles externos proveedores del proceso>	< Información, elementos, materiales, documentos, etc. que provee como insumo del proceso el proveedor correspondiente>
Recursos del proceso	<Se hace especial atención en la definición de los RRHH, también podría enunciarse recursos tecnológicos, tales como software actual o hardware>	
Formularios, registros e información del proceso	Formularios	Registros e información
	<Documentos - Formularios - Salidas Impresas formales>	<Toda información ingresada, procesada y almacenada - también puede considerarse en relación a lo que puede ingresar de información en insumo de proveedores>
Reglas de Negocio	<Políticas, lineamientos, normativas y reglas internas del negocio>	
Restricciones	< Restricciones externas al negocio: leyes, exigencias de organismos reguladores, acuerdos, convenios, etc.>	
Listado de actividades del proceso	< Detalle de actividades realizadas por el curso normal del proceso> [Excepciones] <Todo aquello que interrumpe, cancela o es alternativa del curso normal> Nombre de la excepción <Ej. Anulación de reservas> Descripción: <Breve descripción de lo que comprende la misma>	
Observaciones	<Toda aclaración necesaria del proceso>	

Modelado con BPMN

El objetivo del BPMN **es mejorar la eficiencia** a través de la gestión de los procesos de negocio.

Buscar identificar, diseñar, monitorear, controlar y medir los procesos de negocios que una organización implementa.

Es una **notación gráfica estandarizada**, permite **modelar** procesos de negocio.

Muestra el paso a paso, representando el flujo de extremo a extremo.

Tiene cinco elementos básicos:

1- Objetos de flujo

Actividades:

Representan el trabajo realizado. Transforman el estado de un objeto de negocio para que el proceso pueda llegar a producir valor para los clientes.

Pueden ser simples o compuestas. Si una actividad es atómica y simple se denomina TAREA, y no se puede definir a un nivel más detallado.

Se representan con un rectángulo, y se debe especificar el tipo de TAREA:

Usuario: involucran algún software

Manual: NO involucran ningún software

Servicio: modelan transacciones en línea.

Subproceso

Es una actividad compuesta, significa que es una descripción de un proceso a un nivel intermedio. Se puede desglosar y ver más detalle. Se representa con un rectángulo y un cuadrado con un “+” en la parte central inferior.

Eventos:

Eventos de Inicio → siempre estará, es el disparador de todo.

Eventos intermedios → se pueden usar cuando se necesite modelar algo que pueda ocurrir durante el transcurso del proceso.

Eventos de Fin → siempre estará, es el fin de todo, representa con éxito si se cumplió el objetivo.

Se rotulan con una frase sustantivada, nunca es por ej.: “recibir pedido del cliente”, si no, “pedido de artículos” o “solicitud de artículos”.

Compuertas:

Muestran distintos caminos, ya sea de **apertura** (divergencia) o de **confluir** (convergencia)

Divergencia significa que se inician dos o más caminos en el flujo. Se representa con un rombo.

Convergencia significa que se unen dos o más caminos en el flujo, para continuar con una misma actividad. Se representa con un rombo y un “+” en el medio.

2- Objetos de conexión

Conectan distintos elementos en el diagrama.

Se distinguen dos objetos:

Secuencias → está representada por una flecha y conectan una actividad de inicio con otra.



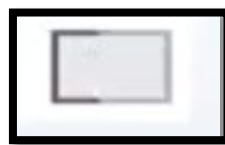
Asociaciones → está representada por una línea de puntos, y asocian información adicional sobre el proceso. Puede ir conectado a un artefacto.



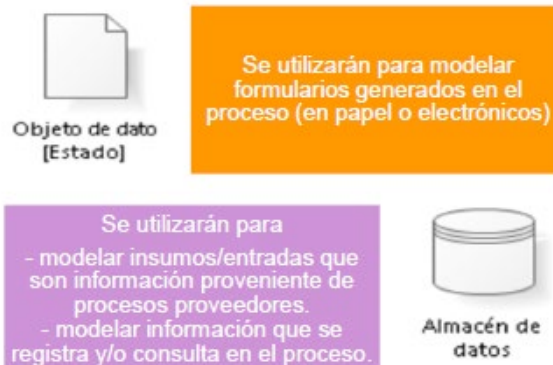
3- Artefactos

Elementos complementarios, opcionales. Se distinguen:

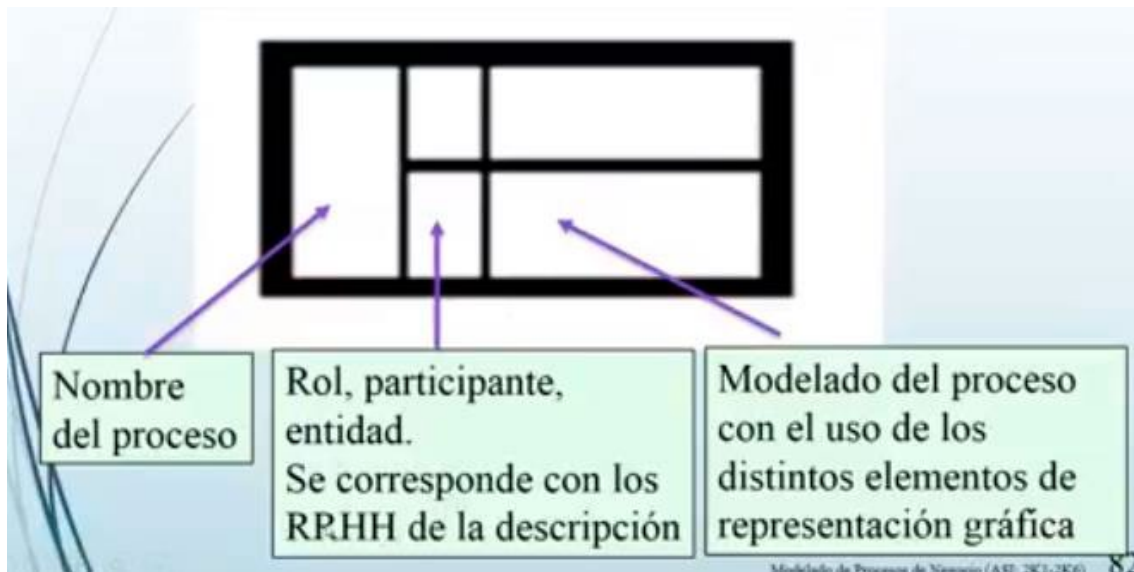
Anotaciones → comentarios, anotaciones o notas que dan información adicional sobre el proceso.



Objeto de datos → información o documentos que son manipulados en un proceso. Pueden ser archivos físicos en papel, documentos, o información electrónica

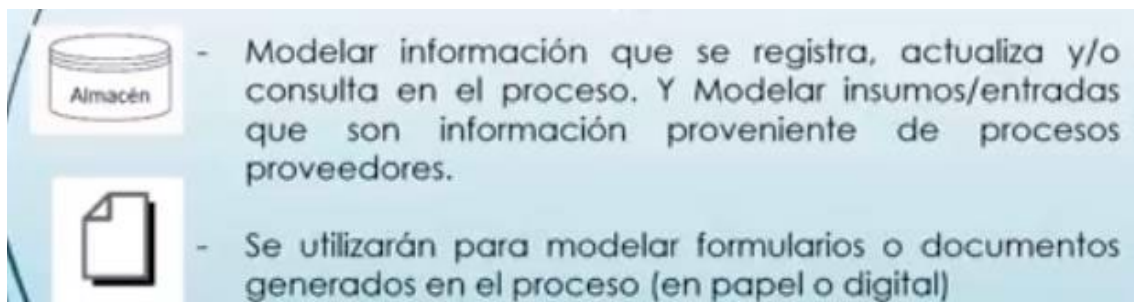


4- Swimlanes (canales o pools)



5- Objetos/almacén de datos

Muestran información que una actividad necesita de entrada y/o salida.



Niveles de granularidad

Hace referencia al nivel de detalle del proceso.

Nivel descriptivo → es el más general, solo muestra subprocessos.

Nivel operativo → es el más descriptivo, muestra con detalles el flujo del proceso.

Unidad 2: Sistemas de información y procesos de desarrollo de sistemas de información y sistemas de software.

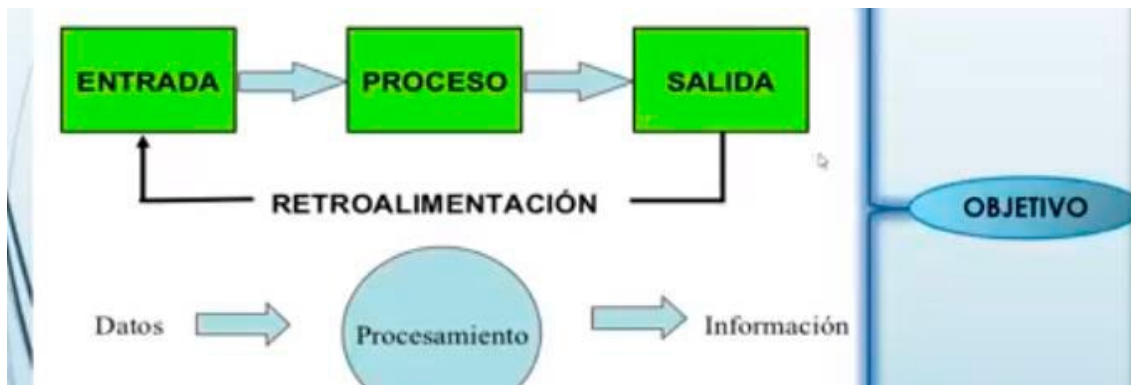
¿Qué es un sistema de información?	12
¿Cómo funcionan los sistemas de información?.....	12
Niveles de toma de decisión y características de la información	13
Tipos de Sistemas de Información	14
Sistemas Transaccionales (profundización)	17
Componentes de un sistema informático	20
Requerimientos	21
Requerimientos funcionales	21
Requerimientos no funcionales	22
Ingeniería de Software	23
Definición	23
Capas de la I.S.....	23
Modelo	25
Proceso de Desarrollo	25
Propósitos del proceso de desarrollo.....	26
Actividades genéricas.....	26
Modelos de Proceso	27
Metodologías Ágiles	30
Proceso Unificado de Desarrollo (PUD)	31
Características esenciales.....	31
Fases y flujos de trabajo de un ciclo.....	32
Conceptos básicos del proceso	32
Estudio de Prefactibilidad	33

Unidad 2: Sistemas de información y procesos de desarrollo de sistemas de información y sistemas de software.

¿Qué es un sistema de información?

Es un **conjunto formal de procesos** que operan sobre una colección de datos que está **estructurada** según las necesidades de la empresa, y distribuyen toda o una parte de la información necesaria para las operaciones y actividades de dirección y control, de acuerdo a la estrategia de negocio.

¿Cómo funcionan los sistemas de información?



Entrada: recopilar, capturar datos primarios que entran al sistema. El ingreso puede ser manual (persona colaborando con el ingreso de datos) o automático (lector de código de barras).

Procesamiento: acciones que ese proceso va a realizar, como conversión de datos, almacenamiento, consultar información, etc.

Salida: se seleccionan las salidas del procesamiento que brindan información útil a la organización, se deben seleccionar los dispositivos que van a emitir esas salidas y en qué formatos van a ser ejecutadas.

Retroalimentación: devolución que sirve para saber que cambios se deben realizar en las actividades, para que funcione más eficientemente.

Niveles de toma de decisión y características de la información



Nivel operativo → **análisis de los resultados** respecto de los **recursos consumidos** en las transacciones, para tomar **decisiones a corto plazo**, y con consecuencias limitadas.

La información es **repetitiva**, se centra en el pasado, sus datos son originados internamente, los reportes están bien estructurados y los datos son bien detallados y precisos.

Nivel táctico → **asigna recursos a mediano plazo**, para mejorar el rendimiento de la empresa.

La información es resumida **estadísticamente y específicamente**.

Nivel estratégico → decide las **líneas que debe seguir** la empresa en el futuro.

La información está en un formato muy resumido, en formato muy variable, y de fuentes externas inesperadas. Las decisiones que se toman están poco formalizadas y están afectadas por cada líder (subjetivismo).

Tipos de Sistemas de Información

Sistemas de Procesamiento de Transacciones (TPS)

Transacción = operación de negocio básica como, por ejemplo, un pedido de un cliente, o una solicitud de compra.

Los TPS son sistemas que atienden a **capturar y procesar** los datos con el detalle necesario, y **ejecutar** cualquier tarea, actualización y consulta de datos acerca de las operaciones esenciales de ese negocio y esa organización. En conclusión, apoyan a los procesos de soporte.

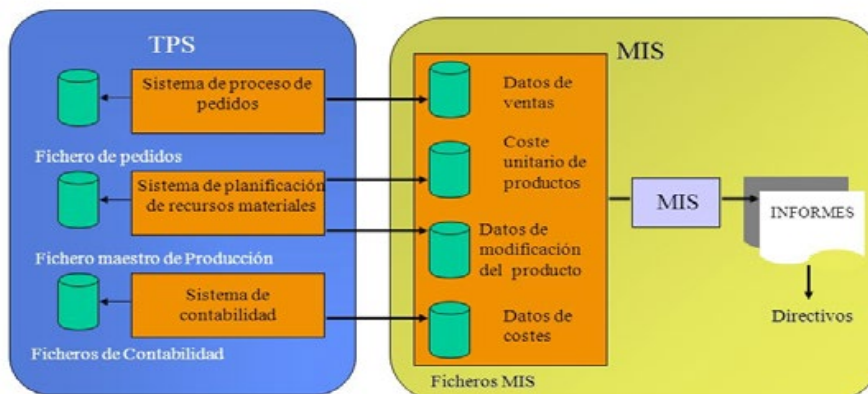
Dan soporte a grandes volúmenes de actividades repetitivas, que tienen datos muy estructuradas y se pueden tratar muy simplemente, que no requieren supervisión humana si se automatizan. **Se optimizan los tiempos y se reducen los costos.**



Sistemas de Información Administrativa (MIS) o Sistema de Información para la Gestión (SIG)

Ejecuta resultados que brindan información que permita tomar decisiones a corto plazo. Por eso este sistema se dedica a apoyar al nivel operativo de la organización.

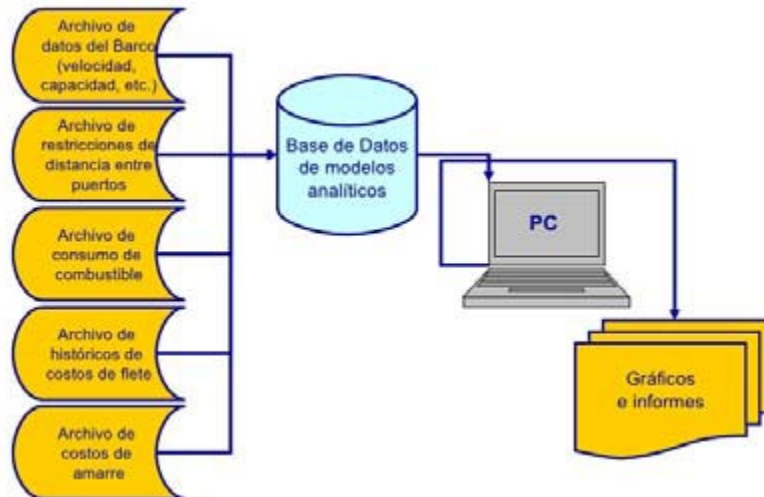
Colabora a la categoría de “decisiones estructuradas”, las cuales generan información estructurada (definida, predeterminada, siempre se hace de esa forma).



Sistemas de Apoyo a la Toma de Decisiones (DSS)

Dan soporte o información que sirva para las decisiones “poco estructuradas”.

Son sistemas de procesamiento que **utilizan información de los TPS**, y emplea datos interactivos que **ofrecen visualización gráfica**, que permiten dar una mejor idea a la hora de tomar decisiones.



Sistemas de Soporte Ejecutivo (ESS)

Se basa en **decisiones poco estructuradas**, se apoya en los TPS, pero está dirigido a los **niveles más altos** de la organización.

Sistema de Gestión del Conocimiento (KWS)

Apoyan a los **trabajadores de conocimientos** (los que crean nuevos productos, diseños, prototipos) y que necesitan un sistema que soporte dicho conocimiento y un software apropiado.

Ayuda a **modelar y simular**, lo que a su vez les permite analizar los problemas y crear nuevos productos.

Inteligencia artificial y sistemas expertos

Están orientados a situaciones que permitan **automatizar** características de la inteligencia humana. Ej.: robótica, redes neuronales.

Tipos de Sistemas de Información

Grupos al que sirven

Sistemas de información ejecutiva (ESS)

Nivel
estratégico

Alta dirección

Sistemas de soporte de decisiones (DSS)
Sistemas de información de gestión (MIS)

Nivel de gestión
y administración

Directivos intermedios

Sistemas de gestión del
conocimiento (KWS)
Sistemas de oficinas (OfficeS)

Nivel de conocimiento

Trabajadores de datos y
conocimiento

Sistemas de procesamiento
de transacciones (TPS)

Nivel operativo

Directivos operativos

Producción y
logística

Finanzas

Recursos
humanos

Administración y
contabilidad

Ventas y
marketing

www.pertutatis.cat

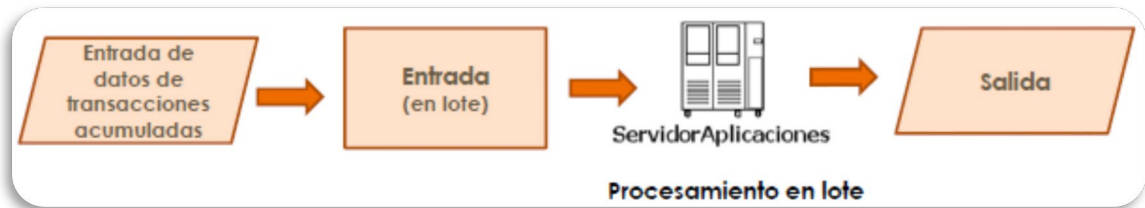
Sistemas Transaccionales (profundización)

Métodos de Procesamiento de Transacciones

En Lotes

Las transacciones quedan agrupadas en “lotes” llamados **batches**, y se procesan todos juntos.

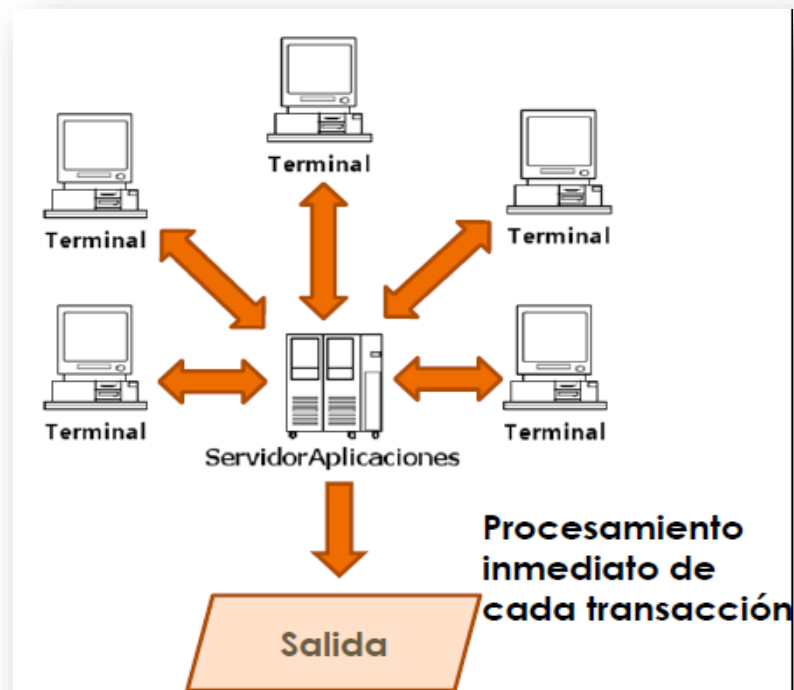
Las transacciones se van acumulando por un periodo y luego se preparan para procesarse todas juntas como un **todo**.



En Línea, de tiempo real (on-line)

Cada transacción se procesa al momento en que se cuenta con la información.

El procesamiento lo realiza una computadora, y actualiza todos los registros implicados en esa transacción.



En línea, con procesamiento retardado

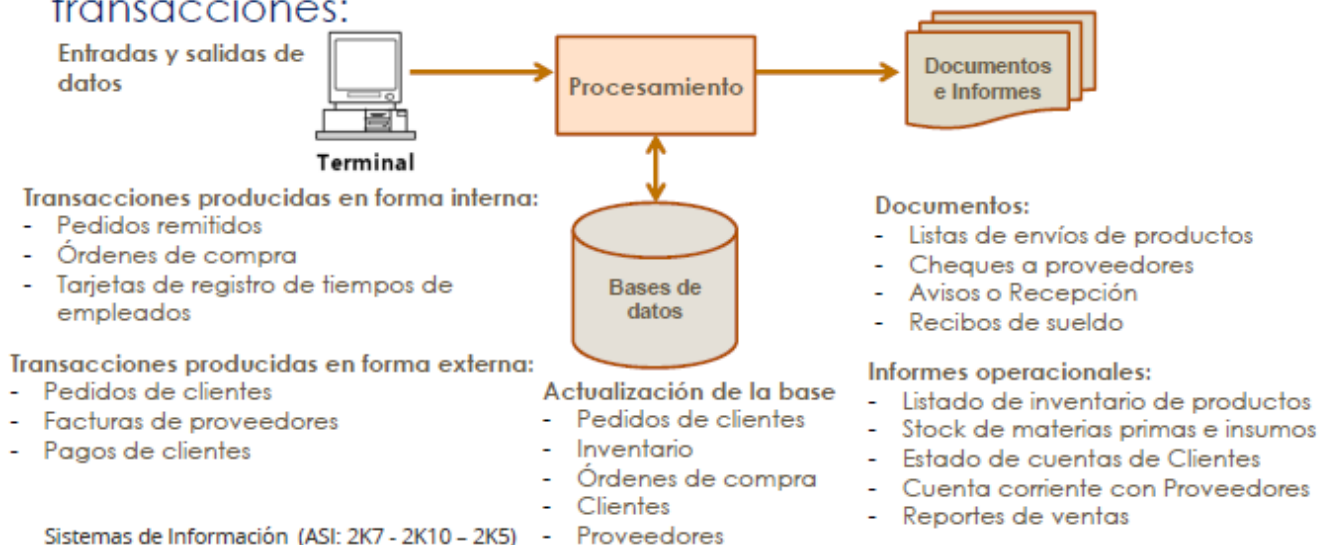
Las transacciones se introducen al sistema en el momento, pero se procesan más tarde.

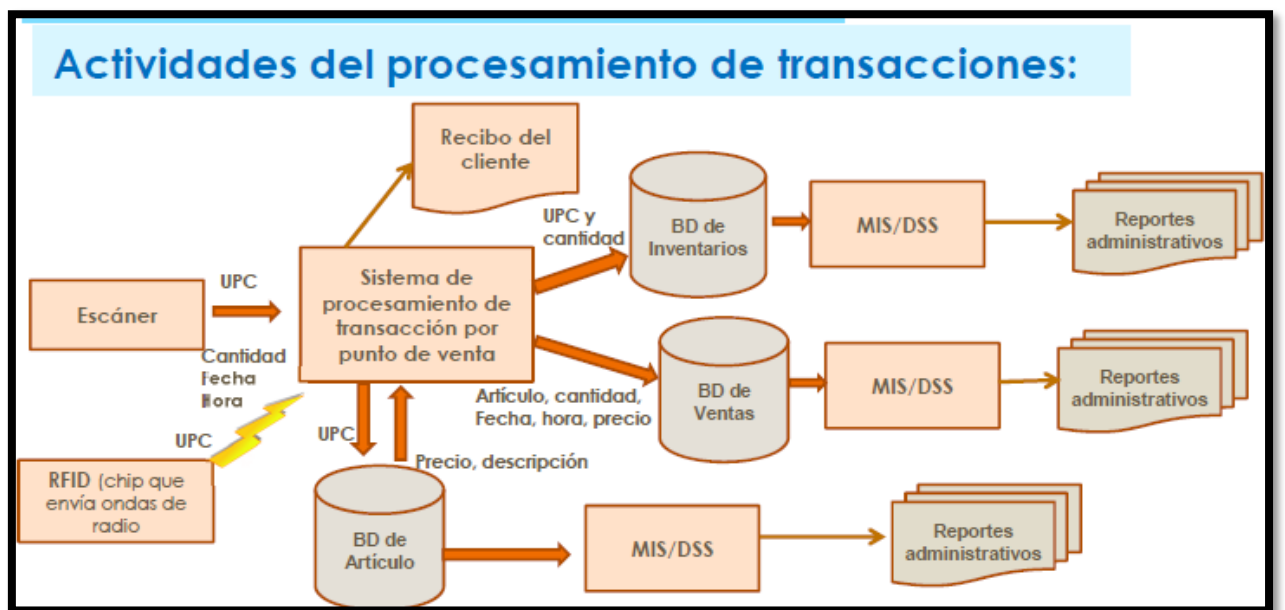
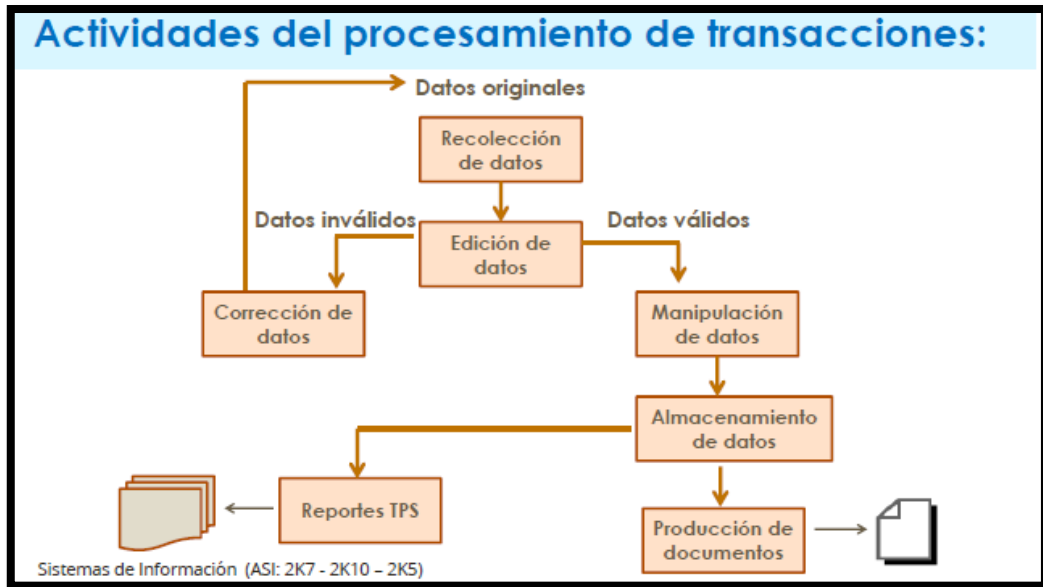
RESUMEN GENERAL DE SISTEMA TRANSACCIONAL

Objetivos:

- Procesar datos generados por las transacciones.
- Mantener un alto grado de exactitud.
- Asegurar integridad y exactitud de los datos y la información.
- Elaborar documentos e informes oportunos.
- Aumentar la eficiencia de la mano de obra
- Ayudar a proporcionar mayores y mejores servicios
- Ayudar a crear y mantener la lealtad del cliente
- Lograr una ventaja competitiva

• Visión simplificada de un sistema de procesamiento de transacciones:





Componentes de un sistema informático

Hardware

Equipamiento de computación y dispositivos varios que se utilizan para llevar a cabo las actividades de entrada, procesamiento y salida.

Software

Programas de computación que dirigen las operaciones de una computadora.

Datos y bases de datos

Conjunto organizado de datos e información, con una estructura que almacena la información de un sistema.

Personas

Individuos que operan, programan y mantienen al sistema.

Procedimientos

Estrategias, políticas, métodos y reglas para el uso del sistema informático

Telecomunicaciones, redes e internet

Transmisiones electrónicas de señales de comunicación que permiten a las organizaciones conectar entre si a sistemas informáticos para formar redes.

A través de internet se logra la interconexión de las redes, las cuales intercambian libremente información.

Requerimientos

Un requerimiento es una característica del sistema o una **descripción de algo que el sistema es capaz de hacer**, con el objetivo de satisfacer el propósito del mismo.

Los requerimientos tienen que tener en cuenta ciertas **restricciones** según necesidades o condiciones del usuario, además de **problemas y mejoras** que se puedan hacer en relación al propio sistema.

Requerimientos funcionales

Definen las **funciones que el sistema es capaz de realizar**, y describen **como se tiene que comportar** el mismo ante determinado estímulo.

Responden al ¿Qué? (qué hace el sistema)

Por lo general, en el listado de requerimientos funcionales, se encuentran los globales, a partir de los cuales luego tenemos, para cada uno, requerimientos funcionales detallados. Ej.:

Ejemplo: El sistema permitirá:

- Administrar datos de clientes
 - Registrar datos del cliente
 - Modificar datos de cliente
 - Consultar datos de clientes
 - Registrar baja de cliente
 - Emitir informe de clientes
- Gestionar ventas de artículos
 - Registrar venta de artículos
 - Generar factura de venta
 - Consultar facturas de venta
 - Anular factura de venta
- Brindar información estadística de ventas
 - Generar informe de ventas e ingresos por periodos
 - Generar estadísticas de artículos mas vendidos

Requerimientos no funcionales

Son las **restricciones de las funciones sobre el sistema**, que justamente **limitan** la posibilidad de elegir una alternativa en la etapa de diseño y construcción del sistema. Por ejemplo: el rendimiento (tiempo y espacio), fiabilidad (disponibilidad del equipo), mantenimiento, seguridad, etc.

Responden al ¿Cómo? (como hace tal cosa el sistema)

Ej.:

El sistema deberá:

- Generar la factura de venta considerando la reglamentación vigente según ley de facturación.
- Generar un informe estadístico de ventas con el logo de la empresa en el borde superior derecho y con gráficos estadísticos a color.
- Tener interfaces con un formato similar a las de Windows.
- Provee entorno Web para las funcionalidades de gestión de pedidos de clientes.

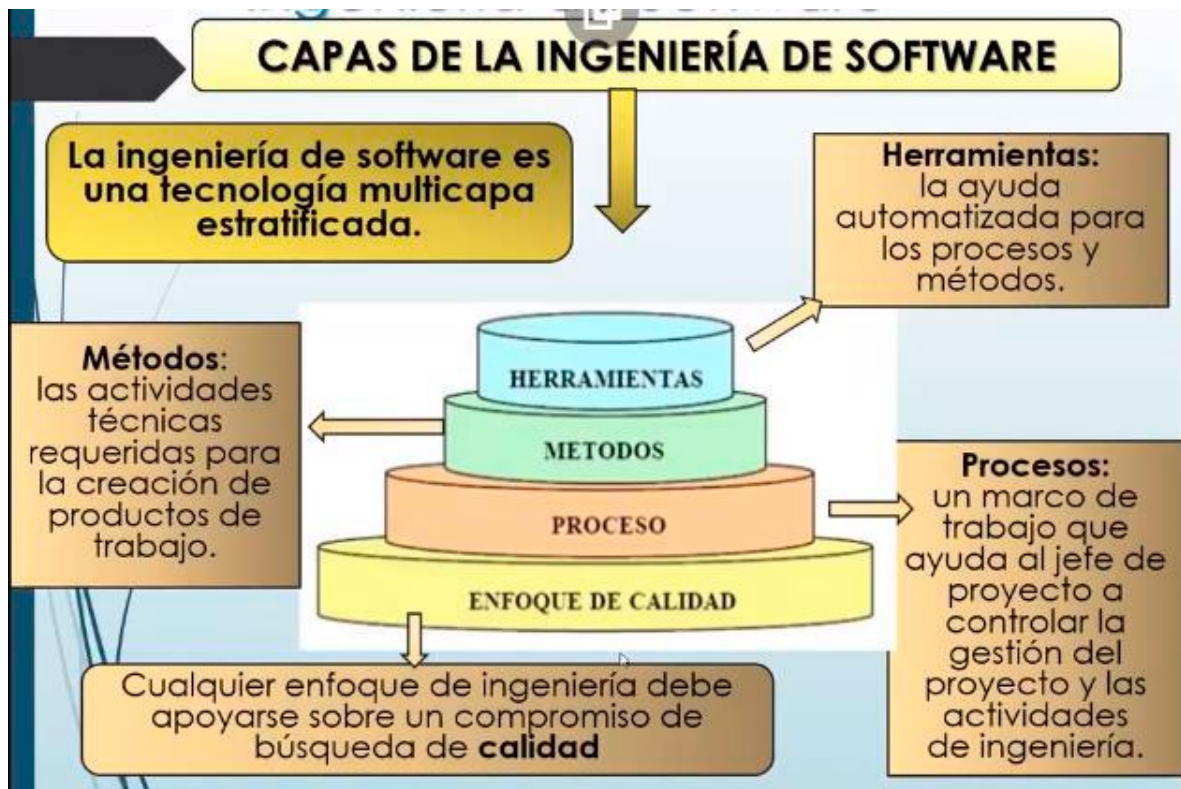
Ingeniería de Software

Definición

Es el establecimiento y uso de **principios robustos** de la ingeniería, con el fin de obtener económicamente algún software que sea confiable y eficiente.

Capas de la I.S

La ingeniería de software es una **tecnología multicapa**. Siempre busca un **enfoque de calidad**.



Proceso

El trabajo de desarrollo está organizado por un marco de trabajo, en el cual **se tienen que definir si o si** todas las actividades que se van a desarrollar. Se decide quién va a hacer cada cosa, cuando, y como lograr cierta meta. Aplica a todo proyecto de software.

Método

Definen **como se va a construir técnicamente un sistema**, teniendo en cuenta la planificación, estimación de proyectos, analizando los requerimientos, y a través de la prueba y el mantenimiento.

Una técnica es un método estructurado y repetible con el fin de lograr una tarea específica.

Herramienta

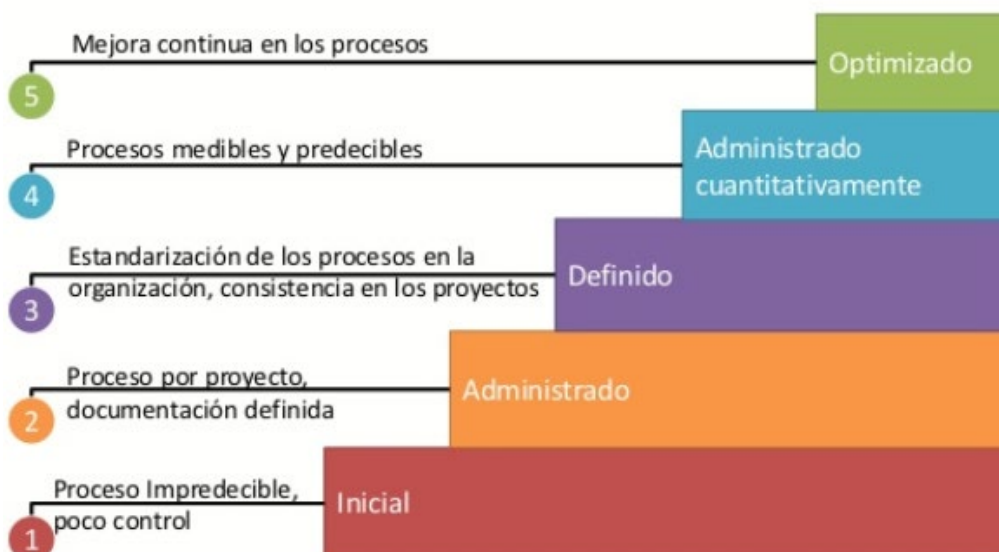
Nos dan **soporte automático o semiautomático**, al proceso y al método.

Las herramientas se conocen como CASE (Ingeniería de Software asistido por computadora), y combinan hardware, software, y bases de datos. Proveen información gráfica acerca del análisis, diseño, y construcción de sistemas.

Calidad

Serie de estándares que certifican que un sistema tiene calidad en desarrollo de software.

A partir de esto surge el **CMM, modelo de capacidad de madurez**, que establece una serie de niveles de acuerdo a la cantidad de estándares cumplidos.



Modelo

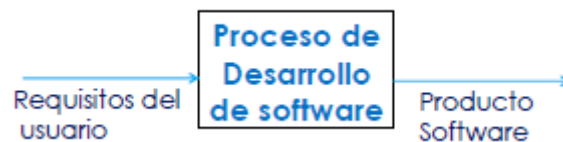
Representación simplificada de la realidad, nos da los planos de un sistema. **Incluye elementos de gran influencia y omite elementos que no son relevantes.**

Los modelos **nos ayudan a ver cómo es o como queremos que sea un sistema.** A su vez, permiten especificar la estructura o el comportamiento del mismo.

Nos dan las plantillas que nos guían en su construcción y documentan las decisiones que se van adoptando.

Proceso de Desarrollo

“Un proceso de desarrollo de software es el conjunto de actividades necesarias para transformar los requisitos de un usuario en un sistema software”



Está conformada por las cuatro “P”:

Personas	Siempre va a haber personas involucradas en el desarrollo del sistema. Son las que financian, planifican, desarrollan, gestionan, prueban y utilizan el producto y obtienen un beneficio.
Proyecto	Es el elemento organizativo por el cual se gestiona el desarrollo del software. El resultado es una versión del producto.
Producto	Son los artefactos que se crean durante el proyecto, ya sea modelos, códigos fuentes, ejecutables y documentación.
Proceso	Definición completa de todas las actividades que son necesarias para transformar los requisitos del usuario en un producto. No se ejecuta, es una planilla para crear proyectos.

Propósitos del proceso de desarrollo

Los desarrolladores del sistema necesitan un proceso que **proporcione una guía** para ordenar las **actividades de un equipo**, que dirija las tareas **de cada uno con su rol** por un lado de forma individual y por otro lado tenemos actividades que se corresponden en **la integración del equipo**.

Tienen que definir cuáles son los **productos a obtener, los artefactos o modelos**, y bajo que paradigma o concepción se va a modelar algo, codificar y crear el producto. Y por último que establezca elementos para **el control de esos productos** para que la gestión de proceso permita decir si voy bien y ajusta lo necesario para llegar a buen término.

Actividades genéricas

En todo proceso de desarrollo de software debe haber:

Definición

Se centra en el QUÉ. Identifica los requisitos claves del S.I detectando:

- ✓ QUÉ información se necesita procesar
- ✓ QUÉ comportamiento y rendimiento se desea
 - ✓ QUÉ interfaces se van a establecer
 - ✓ QUÉ restricciones existen en el diseño
- ✓ QUÉ criterios de validación se necesitan para definir un sistema correcto

Todo esto lo hace a través de dos tareas fundamentales: el **análisis de requisitos** y la **elaboración de modelos**.

Desarrollo

Se centra en el CÓMO:

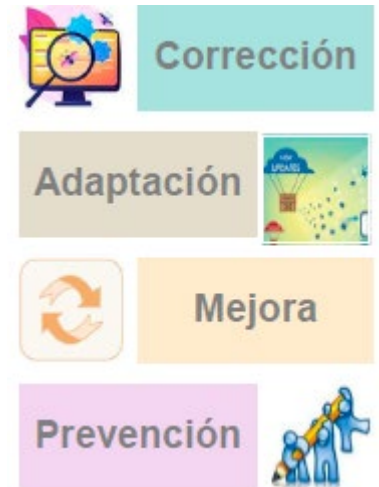
- ✓ CÓMO diseñar las estructuras de datos y la arquitectura del software
 - ✓ CÓMO implementar los detalles más elementales
- ✓ CÓMO traducir el diseño a un lenguaje de programación
 - ✓ CÓMO realizar las pruebas

Todo esto lo hace a través de tres tareas fundamentales: el **diseño del software**, la **codificación** y la **prueba**.

Mantenimiento

Se centra en el CAMBIO. Va asociado a:

- ✓ La corrección de errores y/o adaptaciones por modificaciones en el entorno, o por requisitos cambiantes del usuario.
- ✓ Se vuelven a aplicar los pasos de las fases **definición** y de **desarrollo** ya existentes.



Modelos de Proceso

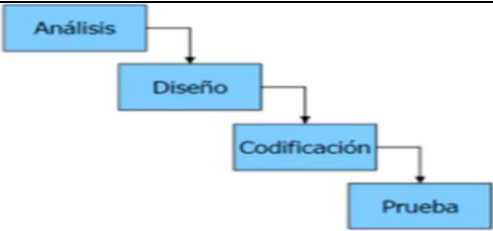
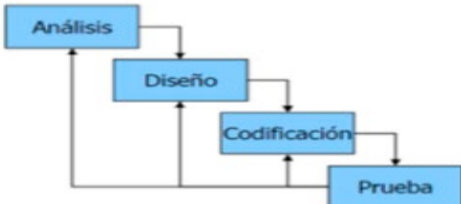
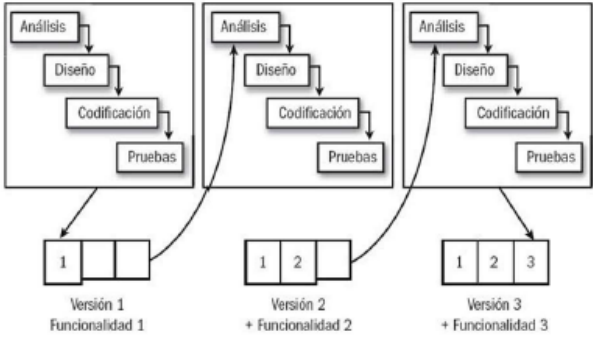
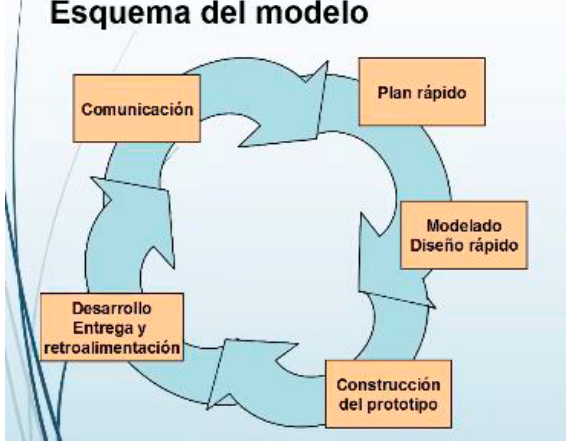
Definen como solucionar la problemática del desarrollo de sistemas de software. Debe considerar varias cosas:

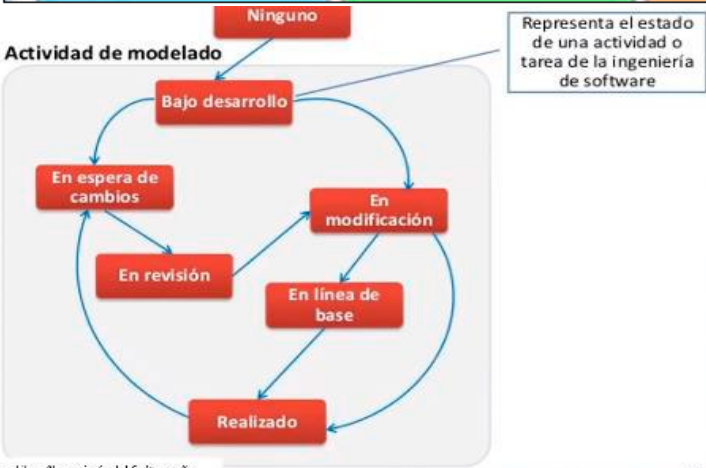
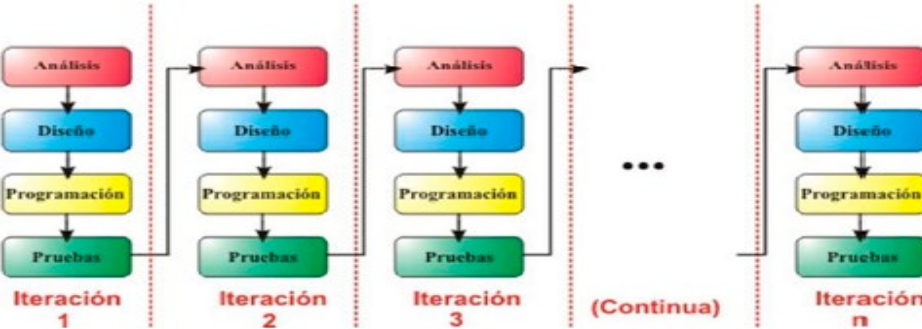
- Los roles que participan
 - Las reglas y políticas
 - Las actividades
- Los componentes del software
- Las metodologías y herramientas utilizadas

Se dividen en dos grupos: los **prescriptivos** y los **adaptativos**

<u>Modelo de Proceso Prescriptivo</u>	Se centran en la identificación y la aplicación detallada de las actividades del proceso. El objetivo es mejorar la calidad del sistema, hacer proyectos más manejables, y hacer más predecibles las fechas de entrega y los costos.
<u>Modelo de Proceso Adaptativo (Ágiles)</u>	Se centran en la agilidad del proyecto, se busca un enfoque del proceso de software más liviano y menos complejo. Se dice que son “ágiles” porque acentúan la maniobrabilidad y la adaptabilidad.

Tipos de modelos descriptivos

Modelo en cascada	 Ciclo de vida clásico	Primero en surgir Tipo Lineal Enfoque sistemático y secuencial Una vez se termina una actividad no se vuelve para atrás.
Modelo en cascada con realimentación	 Ciclo de vida clásico con realimentación	
Modelo incremental		Permite producir software en incrementos, en los que en cada uno de éstos se entrega un producto que ya opera. Brinda así progresivamente mayor funcionalidad al cliente.
Modelo en V		Trabaja en una secuencia de cascada, pero establece diversas pruebas durante la ejecución de ese sistema. Es por eso que se dice que amortigua al modelo en cascada.
Modelo de procesos evolutivos	Esquema del modelo 	El sistema se desarrolla en iteraciones hasta llegar a una versión final. Es un modelo evolutivo ya que va a agregando nuevos atributos propuestos por el cliente.

Modelo en espiral		Combina el modelo en cascada con el modelo incremental, y la construcción de prototipos. Comienza desde el centro y pasa por las actividades genéricas. En cada vuelta hace una iteración (incremento) mientras diseña prototipos, a la vez que consulta con el cliente. En cada vuelta del espiral se va creando una nueva y mejorada versión del sistema hasta finalmente culminar.
Modelo especializado de procesos	 DESARROLLO BASADO EN COMPONENTES: El modelo del desarrollo basado en componentes lleva a la reutilización del software, y eso da a los ingenieros de software varios beneficios en cuanto a la mensurabilidad. Si la reutilización de componentes se vuelve parte de la cultura, el equipo de ingeniería de software tiene la posibilidad tanto de reducir el ciclo de tiempo del desarrollo como el costo del proyecto.  EL MODELO DE MÉTODOS FORMALES: El modelo de métodos formales agrupa actividades que llevan a la especificación matemática formal del software de cómputo. El enfoque de los métodos formales ha ganado partidarios entre los desarrolladores que deben construir software de primera calidad en seguridad (por ejemplo, control electrónico de aeronaves y equipos médicos).  DESARROLLO DE SOFTWARE ORIENTADO A ASPECTOS: Es un paradigma de ingeniería de software relativamente nuevo que proporciona un proceso y enfoque metodológico para definir, especificar, diseñar y construir aspectos: "mecanismos más allá de subrutinas y herencia para localizar la expresión de una preocupación global".	
Modelo de desarrollo concurrente	 Actividad de modelado Ninguno Bajo desarrollo En espera de cambios En modificación En revisión En línea de base Realizado Representa el estado de una actividad o tarea de la ingeniería de software	Por cada actividad se lleva una secuencia de control del estado. No es tan utilizado.
Modelos iterativos	 Análisis Diseño Programación Pruebas Iteración 1 Iteración 2 Iteración 3 (Continúa) Iteración n	

Metodologías Ágiles

Nueva corriente que se separó de los modelos prescriptivos. Surge a partir de los inconvenientes de los modelos clásicos. **Propone una nueva forma de trabajar.**

Sirve para desarrollar sistemas donde los requerimientos de los clientes no están tan claros desde el inicio y pueden ir cambiando. Es por eso que en el desarrollo del sistema **existe mucha interacción con el cliente, se reduce la cantidad de documentación, y directamente se centra en el resultado para poder validarlo conforme avanza**, para darle al cliente un resultado al cliente de calidad.

Proceso Ágil

- ✓ Debe ser adaptable **en forma incremental** a un proyecto y a condiciones técnicas que cambian rápidamente.
- ✓ Los incrementos de software tienen que entregarse en **cortos periodos** para que la adaptación se mantenga en ritmo.
- ✓ **Permite al cliente evaluar el incremento de software de manera regular**, de manera que proporcione retroalimentación al equipo, e influir sobre las adaptaciones del proceso.
- ✓ No se tiene certeza sobre cuales requerimientos de software permanecerán o cambiarán, tampoco sobre las prioridades del cliente a medida que avanza el proyecto.
- ✓ Para muchos, el diseño y la construcción se ejecutan en forma simultánea, esto quiere decir que los modelos de diseño se prueben a medida que se van creando. Estos 2 procesos tampoco son tan predecibles (desde un punto de vista de planeación)

Proceso Unificado de Desarrollo (PUD)

Es un proceso de desarrollo de software que utiliza el **Lenguaje Unificado de Modelado (UML)** para preparar todos los esquemas de un sistema de software. Este modelo se basa en el paradigma orientado a objetos (POO) y se utiliza para preparar todos los modelos de un software.

Se puede definir como el conjunto completo de actividades, métodos, fundamentos y practicas necesarias para **transformar los requisitos de un usuario**, en un conjunto consistente de artefactos (producto) que conforma un **producto de software**.

Características esenciales

Dirigidos por casos de uso

Un caso de uso es un fragmento de funcionalidad que el sistema va a considerar, y que le va a otorgar al usuario un resultado importante para cubrir una función.

Se van a considerar los requerimientos funcionales.

Su funcionamiento se basa en **capturar requerimientos funcionales** y realizar iteraciones, en las que en cada una se tome un conjunto de casos de uso y se desarrolle el camino a través de distintas disciplinas (diseño, implementación, prueba, etc.)

Centrado en la arquitectura

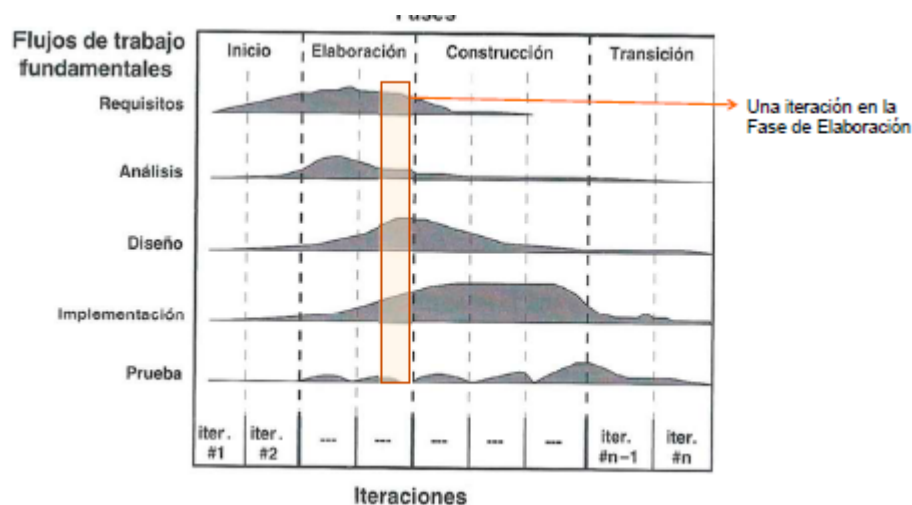
El PUD asume que **no hay un modelo único para cubrir todos los aspectos del sistema**, es por eso que existen múltiples modelos y vistas que definen a la arquitectura de software de un sistema.

Iterativo e incremental (vida del proceso unificado)

Es un **marco de desarrollo iterativo e incremental compuesto de cuatro fases**: Inicio, Elaboración, Construcción, y Transición. Cada una de estas fases es a su vez dividida en una serie de iteraciones, las cuales **ofrecen como resultado un incremento del producto desarrollado**, que añade o mejora las funcionalidades del sistema en desarrollo.

A su vez, cada una de estas iteraciones se divide en una serie de disciplinas: Análisis de Requisitos, Diseño, Implementación, y Prueba.

Fases y flujos de trabajo de un ciclo



En el flujo de trabajo se establece qué actividades hay que hacer dentro del mismo, y que artefactos se logran producir. En el grafico se muestra el esfuerzo de cada flujo de trabajo en cada fase.

Inicio: trabajo de planificación, se define una lista de requerimientos

Elaboración: creación de modelos y estructura lógica

Construcción: programación

Transición: puesta en marcha del sistema.

Conceptos básicos del proceso

Flujo de trabajo (Workflow): conjunto de actividades referidas al desarrollo de un sistema, más bien referidas a un aspecto del mismo en particular (requerimientos, análisis, diseño), estableciendo a los trabajadores que participan, las actividades a realizar y los artefactos que se producen.

Trabajador: rol de persona responsable de realizar actividades y producir artefactos

Actividades: distintas tareas a realizar en cada flujo de trabajo para realizar los artefactos necesarios.

Artefacto: resultado que se produce o elabora en un flujo de trabajo.

Estudio de Prefactibilidad

Factibilidad: significa que el proyecto propuesto logre sus objetivos generales a través de la organización de tres áreas:

Factibilidad Técnica

El analista tiene que determinar si los recursos técnicos actuales tienen que ser mejorados o añadidos para satisfacer la petición bajo consideración. Si no se puede, se cuestiona si existe tecnología para hacerlo. Si la respuesta es SI, entonces se procede a la Factibilidad Económica

Factibilidad Económica

Si los costos a corto plazo no son sobrepasados por las ganancias a largo plazo, **entonces el sistema no es factible económicamente** y el proyecto no debería continuar.

Se realiza una identificación y estimación de Costos y Beneficios.

Costos	Desc.	Beneficios	Desc.
Costo del dinero	Préstamos	Tangibles	Ventajas medibles en pesos que se acumulan en la organización. Ej.: reducción de personal, mayor velocidad de procesamiento de transacciones, reducción de costos, etc. Se pueden medir en pesos, recursos o tiempo ahorrados .
Costos operacionales	Materiales, equipo, software, personal, mantenimiento		
Costos de construir el sistema	Salarios y gastos del personal.	Intangibles	Posibilidad de realizar cosas que no eran posibles anteriormente. Ej.: mejorar en la toma de decisiones, atraer nuevos clientes, entrar a nuevos mercados...
Costos de instalar el sistema	Capacitaciones de usuarios, aprobación reglamentaria, instalación comercial		
Costo del fracaso	Posibles fallas		

Factibilidad Operativa

Depende de los recursos humanos disponibles para el proyecto, e involucra proyectar si el sistema operará y será utilizado una vez esté instalado.

Analiza aspectos referentes de los futuros usuarios del sistema, básicamente define como se va a hacer uso del sistema y como se debe diseñar las ventanas del mismo.

Unidad 3: Paradigma Orientado a Objetos

Concepto	35
Complejidad del software	35
Características	36
Objeto.....	36
Clase	37
Elementos esenciales de una clase	38
Relaciones entre clases	39
Diagrama de clases.....	40
Relaciones entre objetos.....	40
Modelo de Dominio	41
Patrones para Modelo de Dominio	41
UML.....	43
Objetivos	43
UML 2.0	43
Estructura UML 2.0	44
Elementos estructurales.....	45
Elementos de comportamiento	45
Elementos de agrupación y anotación	46
Diagramas de estructura	46
Diagramas de Comportamiento	47
Reglas de UML.....	48
Vistas de un sistema con UML	48

Unidad 3: Paradigma Orientado a Objetos

Concepto

“Un paradigma es un concepto de teorías, estándares y métodos, que juntos representan una forma de organizar el conocimiento, es decir, una forma de ver el mundo”

Complejidad del software

Surge de la “crisis del software”, durante la cual **los clientes no lograban la satisfacción por los productos esperados**, o los profesionales **no lograban cumplir con su objetivo** esperado por el cliente. Ésta serie de errores no los cometían a propósito, si no que la tecnología presente no fue capaz. El software no estaba ajustado a la evolución.

Es por eso que ésta crisis necesitaba un tratamiento de la complejidad, que fue la que llevo al software a derivar en la complejidad del dominio del problema.

La técnica para dominar la complejidad de un sistema **es descomponerlo en partes más y más pequeñas**, cada una de las cuales se puede refinar en forma independiente. Las dos formas para descomponer un sistema complejo son:

Descomposición Algorítmica	✓ Se divide al sistema en elementos funcionales, que están relacionados estructuralmente entre sí. ✓ Se pone énfasis en el orden de los eventos. ✓ Los datos y/o funciones son visibles y accesibles en todo el sistema.
Descomposición Orientada a Objetos	✓ Establece un conjunto de agentes autónomos (objetos) que colaboran para llevar a cabo algún comportamiento. ✓ Los datos y el comportamiento se modelan juntos, y son encapsulados.

Características

1. Permite organizar al sistema de acuerdo con abstracciones de más alto nivel, siendo éstas las más cercanas a la manera de pensar de la gente.
2. Los sistemas se suelen construir a partir de objetos ya existentes.
3. La complejidad de los objetos que se pueden utilizar sigue aumentando.
4. Los datos globales desaparecen, son parte interna de los objetos junto con las funciones, por lo que los cambios solamente afectan al objeto.

Objeto

Un objeto es cualquier cosa que tenga **estructura** y **comportamiento**.

Una definición más formal: un objeto **representa** un elemento, unidad o entidad **individual e identificable**, ya sea **real o abstracta**, con un papel bien definido en el dominio del problema.

¡TODO OBJETO TIENE UN ESTADO, UN COMPORTAMIENTO, Y UNA IDENTIDAD!

Estado	Comportamiento	Identidad
Abarca todas las propiedades (normalmente estáticas) del mismo, más los valores actuales (normalmente dinámicos) de cada una de esas propiedades. Ej.: una caja de ahorro tiene varios estados, abierta, cerrada, sin saldo, con saldo.	Es cómo actúa y reacciona un objeto, en termino de sus cambios de estado y paso de mensajes. Su actividad es visible y comprobable exteriormente. Ej.: una caja de ahorro puede incrementar su saldo, disminuir su saldo, mostrar saldo, mostrar titular, etc. Los tres tipos de comportamientos son: <u>Modificador:</u> operación que altera el estado de un objeto <u>Selector:</u> operación que accede al estado de un objeto, pero no lo altera. <u>Iterador:</u> operación que permite acceder a todas las partes de un objeto en algún orden establecido.	Es la propiedad de un objeto que lo distingue de los demás objetos. Ej.: aunque dos manzanas tengan el mismo tamaño y color, siguen siendo diferentes objetos.

Clase

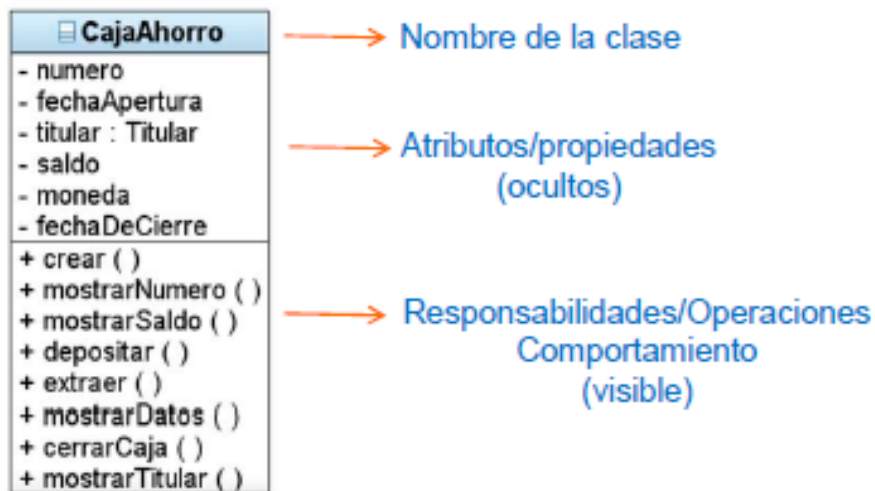
“Conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica”

Una clase puede representar → roles (de personas u organizaciones), lugares, cosas, conceptos, eventos/transacciones.

La naturaleza de una clase comprende dos vistas:

1. **Vista Externa:** Declaración de todas las operaciones (comportamiento) que son aplicables a todas las instancias de la clase (objetos) → **INTERFAZ**
2. **Vista Interna:** Implementación de las operaciones definidas en su interfaz.

Ejemplo de una clase en UML:



Elementos esenciales de una clase

Abstracción	Denota las características esenciales de un objeto que los distingue de todos los demás tipos de objetos, y proporciona fronteras conceptuales nítidamente definidas respecto a la perspectiva del observador.
Encapsulamiento	Proceso de almacenar en un mismo comportamiento a los elementos de una abstracción, y su posterior implementación. Se busca ocultar algo, precisamente los detalles para poder facilitar el manejo de la complejidad, conociendo solamente el comportamiento del objeto y no sus detalles internos.
Modularidad	Forma de agrupar o empaquetar abstracciones en unidades discretas. Al tener mucha cantidad de clases, la mejor forma de manejarlas es agrupándolas para cumplir dos conceptos: el bajo acoplamiento (menor dependencia entre clases y sus objetos) y una alta cohesión (cada definición de clase posea responsabilidades que le sean propias)
Jerarquía	Clasificación u ordenación de abstracciones. Sirve para manejar la interpretación de un dominio y poder comprenderlo mejor.

Relaciones entre clases

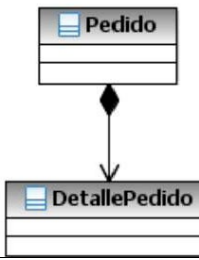
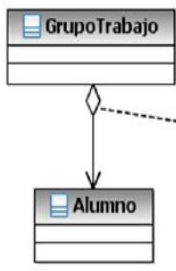
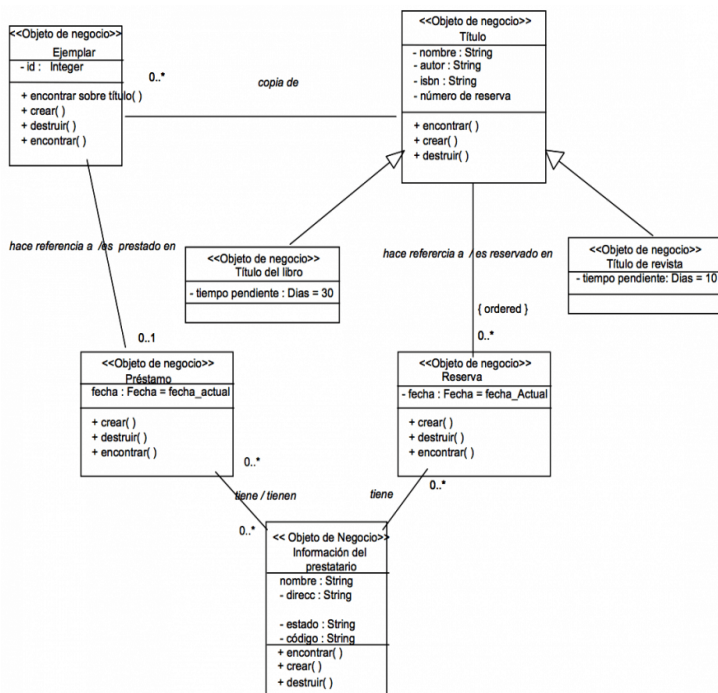
Relación	Descripción	Simbología
Generalización / Herencia	Clase superior o inferior. De la superior (padre) heredo comportamientos y agrego los propios, y de la inferior (hijo) los cedo (y ésta hace lo mismo).	
Asociación	Especifica que los objetos de una clase están conectados con los objetos de otra. Sólo denota una dependencia semántica, no establece la forma exacta en que una clase se relaciona con la otra. Va acompañada con la multiplicidad, la cual indica cuántos objetos pueden conectarse a través de una instancia. Ej.: 1.. * = de 1 a muchos 1 = 1 solo 0 ... 1 = de 0 a 1 1 ... 3 = de 1 a 3 Si la multiplicidad es 0, la relación es opcional, y si es de 1 o más, es obligatoria	
Composición	Representa una relación que significa que un objeto del todo tiene objetos de la parte .	 
Agregación	Relación en donde el tiempo de vida del objeto incluido (la parte) es independiente del que lo incluye.	 Un grupo se puede disolver (se elimina ese objeto todo/contenedor) pero el o los alumnos (parte) que lo formaban siguen existiendo y pasaran a formar parte de otro objeto grupo. 
Dependencia	Indica que una clase utiliza a otra como argumento en la signatura de una operación. Ej.: mostrar la relación entre un cliente y el proveedor de un servicio usado por el cliente.	

Diagrama de clases

Muestran al **conjunto de clases y a sus relaciones**.

Se pueden encontrar → clases, relaciones, indicadores de multiplicidad y navegabilidad y nombre de roles.



Para encontrar clases:

- ✓ Buscar en los requerimientos frases sustantivas y hacer una lista preliminar
- ✓ Dividir esa lista en tres categorías: clases obvias, clases absurdas, y clases dudosas.
- ✓ Ver el rol que tienen éstas abstracciones, pueden ser que sean: cosas tangibles, roles desempeñados, hechos, o transacciones.

Para encontrar responsabilidades de las clases:

- ✓ Si mantienen el conocimiento de un objeto → Conocer
- ✓ Si son acciones que puede ejecutar un objeto → Hacer

Relaciones entre objetos

Relación	Descripción	Simbología
Enlace	Conexión física o conceptual. Un objeto (cliente) utiliza los servicios de otro objeto (servidor). Un objeto puede comunicarse con otro.	
Agregación	Denota una jerarquía del tipo todo/parte, en la cual un objeto tiene objetos de la parte.	

Modelo de Dominio

El Modelo de Dominio **captura los tipos de objetos más importantes** en el contexto del sistema.

Los **objetos del dominio** representan las “cosas” que existen o los “eventos” que suceden en el entorno en el que se desenvuelve el sistema.

Las **clases del dominio** del sistema se pueden deducir a partir de: la especificación de los requisitos, las entrevistas con los expertos del dominio, o bien del Modelo del Negocio.

Las **clases del dominio** que se encuentren se utilizarán al describir los casos de uso, al diseñar el prototipo de la interfaz de usuario, y para sugerir clases internas al sistema en desarrollo durante el Análisis.

La **utilidad** de este modelo es justamente **ayudar** a los usuarios, desarrolladores y otros participantes del proyecto a utilizar un vocabulario común.

Pautas

- Solo debe contener elementos esenciales
- Debe proporcionar detalles consistentes en cuanto al nivel de abstracción
- Debe distribuir elementos de modo de minimizar los cruces de líneas
- Organizar los elementos de modo que sean semántica y físicamente cercanos

Patrones para Modelo de Dominio

Un patrón de software es una **descripción de un problema** que ocurre y de sus posibles soluciones.

Permiten reusar soluciones de análisis o diseño.

Un patrón de modelo de objetos es una agrupación de objetos con responsabilidades estereotipadas y escenarios de interacción.

Los patrones nos ayudan a encontrar las clases, sus relaciones, atributos y responsabilidades

Tipos de patrones

Tipo	Descripción	Ejemplo
Patrón Fundamental	Es la plantilla que todos los patrones siguen, ayuda a encontrar responsabilidades y atributos, las relaciones y su tipo, y que multiplicidad puede existir.	
Patrones Transaccionales	Son los siguientes: #2 Actor – Participante #3 Participante – Transacción #4 Lugar – Transacción #5 ItemEspecífico – Transacción #6 Transacción – Detalle Transacción #7 Transacción – Transacción subsiguiente #8 Detalle Transacción-DetalleTrSubsiguiente #9 Item – Detalle Transacción #10 Item Específico – Detalle Transacción #11 Item – Item Específico #12 Asociación – Otra asociación #13 Item Específico – Item Jerárquico	
Patrones de agregación	Son los siguientes: #14 Contenedor-Contenido. #15 Contenedor-Detalle de Contenedor #16 Grupo-Miembro #17 Todo-Parte #18 Compuesto de Parte -Parte #19 Paquete-Componente de Paquete	
Patrones de plan	Son los siguientes: # 20 Plan-Paso. # 21 Plan-Ejecución de plan #22 Paso-Ejecución de Paso #23 Ejecución de Plan –Ejecución de paso # 24 Plan-Versión de plan	de Plan:

UML

El Lenguaje **Unificado** de **Modelado** es un lenguaje estándar para escribir “planos” de software.

Un lenguaje proporciona un vocabulario y las reglas para combinar palabras de ese vocabulario con el objetivo de posibilitar la comunicación.

Un lenguaje de modelado es un lenguaje cuyo vocabulario y reglas se centran en la representación conceptual y física de un sistema.

Objetivos

Visualizar	Permite crear gráficamente una serie de diagramas que permite mostrar el sistema.
Especificar	Describe la estructura o comportamiento de un sistema de forma precisa, completa, y sin ambigüedades.
Documentar	Guardar las decisiones que se van tomando y construir la documentación de un sistema.
Construir	Transitar desde los modelos a la construcción del sistema, a la codificación

UML 2.0

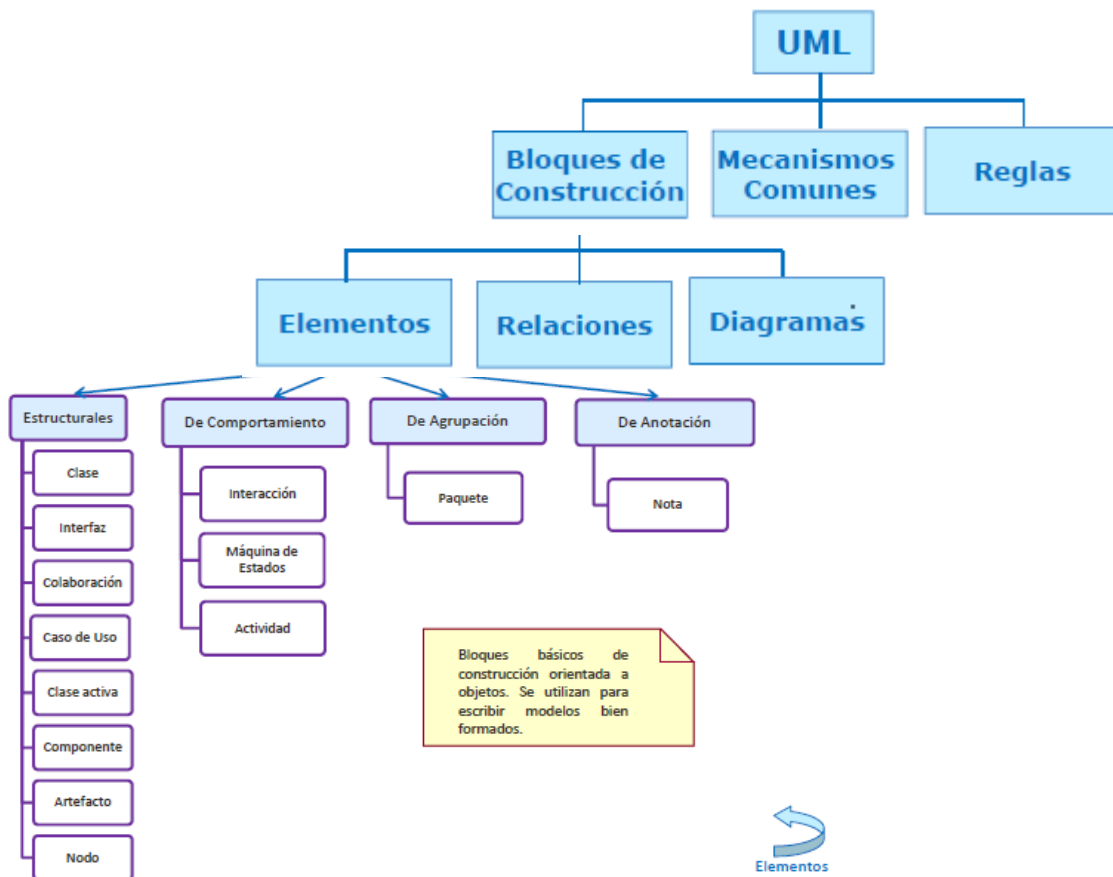
Los **objetivos** de esta nueva versión son:

- ✓ Hacer el lenguaje de modelado más extensible.
- ✓ Permitir la validación y ejecución de modelos.

Los **cambios** de esta nueva versión son:

- ✓ Numerosa **sintaxis visual nueva**.
- ✓ Numerosos **cambios sintácticos** → capturan mucho más comportamiento, dando lugar a herramientas que soporten la automatización y generación de código ejecutable a partir de modelos UML.

Estructura UML 2.0



Bloques de construcción: conjunto de elementos, relaciones y diagramas que permiten crear los distintos modelos.

Mecanismos Comunes: establece patrones de características comunes.

Reglas: especifican a que debe parecerse un modelo bien formado (que sea semánticamente autoconsistente y en armonía con sus modelos relacionados)

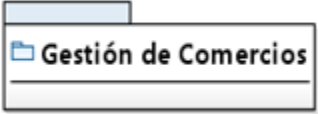
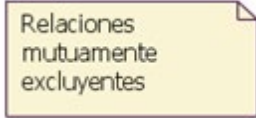
Elementos estructurales

Elemento	Descripción	Imagen
Clase	Conjunto de objetos con los mismos atributos, operaciones, relaciones y semántica	
Interfaz	Representa el comportamiento de una clase.	
Colaboración	Representa una interacción y es una sociedad de roles y otros elementos que colaboran para lograr un comportamiento cooperativo mayor que la suma de comportamientos de sus elementos.	
Caso de Uso	Conjunto de secuencias de acciones que un sistema ejecuta y que produce un resultado observable de interés para un actor particular.	
Clase activa	Clase cuyos objetos tienen uno o mas procesos o hilos de ejecución y por lo tanto pueden dar origen a actividades de control.	
Componente	Parte modular del diseño del sistema que oculta su implementación tras un conjunto de interfaces externas. Puede expresarse conectando partes y conectores.	
Artefacto	Parte física y reemplazable de un sistema que contiene información física.	
Nodo	Recurso físico que existen en tiempo de ejecución y representa un recurso computacional, dispone de memoria y capacidad de almacenamiento. Un conjunto de artefactos puede residir en un nodo.	

Elementos de comportamiento

Elemento	Descripción	Imagen
Interacción	Comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos.	
Máquina de Estados	Comportamiento que especifica las secuencias de estados por las que pasa un objeto o una interacción durante su vida.	
Actividad	Comportamiento que especifica la secuencia de pasos que ejecuta un proceso computacional.	

Elementos de agrupación y anotación

Elemento	Descripción	Imagen
Paquete (Agrupación)	Representan las cajas en las que puede descomponerse un modelo. Los paquetes organizan los elementos en grupos.	
Nota (Anotación)	Son comentarios que se pueden aplicar para hacer observaciones sobre cualquier elemento del modelo.	

Diagramas de estructura

Representan elementos y así componen un sistema o una función.

Estos diagramas pueden reflejar:

- Las relaciones estáticas de una estructura.
- Arquitecturas en tiempo de ejecución.



Diagrama	Descripción
Diagramas de Clase	Muestra los bloques de construcción. Describen la vista estática de una parte o de todo el modelo. Ilustran relaciones entre clases e interfaces.
Diagramas de Objetos	Caso especial de un diagrama de clase. Enfatizan sobre la relación entre las instancias de clases en algún momento en el tiempo. Reflejan multiplicidad y roles.
Diagramas de Estructura Compuesta	Representan patrones de diseño, muestra la estructura interna de un clasificador, incluyendo sus puntos de interacción a otras partes del sistema. Los conceptos para describir la estructura interna son: Partes, Puertos y Conectores.
Diagrama de Componentes	Muestran las piezas de software, controladores, etc., que componen un sistema. Un componente es ejecutado por una o mas clases u objetos en tiempo de ejecución.
Diagrama de Despliegue	Modela la arquitectura en tiempo de ejecución. Muestra la configuración de los elementos de hardware (nodos) y como los elementos de software y los artefactos se asignan en los nodos.
Diagrama de Paquetes	Se usan para reflejar la organización de paquetes y sus elementos. Sus usos mas comunes son para organizar diagramas de casos de uso y diagramas de clase.

Diagramas de Comportamiento

Representan las características de comportamiento de un sistema.



Diagrama	Descripción
Diagrama de Actividad	Muestra la secuencia de actividades, específicamente el flujo de trabajo desde el punto de inicio hasta el punto final, detallando muchas de las rutas de decisiones que existen en el progreso.
Diagrama de Casos de Uso	Captura los requisitos de un sistema. Muestra la interacción entre el sistema y las entidades externas al sistema (agentes).
Diagrama de Máquina de Estados	Modela el comportamiento de un objeto especificando la secuencia de situaciones que atraviesa durante su ciclo de vida en respuesta a eventos.
Diagrama de Descripción de Interacción	Forma de un diagrama de actividad en la cual los nodos representan diagramas de interacción.
Diagrama de Comunicación	Diagrama de interacción que muestra información enfocada en las relaciones entre objetos.
Diagrama de Secuencia	Muestra cuales objetos se comunican con otros objetos, y qué mensajes disparan esas comunicaciones.
Diagrama de Tiempos	Mostrar cambios en el estado de una línea de vida de una instancia a lo largo del tiempo, de manera lineal.

Reglas de UML

UML tiene reglas semánticas para:



- **Nombres:** Cómo llamar a los elementos, relaciones y diagramas.
- **Alcance:** El contexto que da un significado específico a un nombre.
- **Visibilidad:** Cómo se pueden ver y utilizar esos nombres por otros.
- **Integridad:** Cómo se relacionan apropiada y consistentemente unos elementos con otros.
- **Ejecución:** Qué significa ejecutar o simular un modelo dinámico.

Vistas de un sistema con UML

Vista	Descripción	Aspectos
Vista de Casos de Uso	Describir comportamiento a través de casos de uso.	Estático: diagrama de casos de uso Dinámico: diagramas de interacción, de estados y de actividades.
Vista de Diseño	Clases, interfaces y colaboraciones que conforman el vocabulario del sistema y la solución.	Estático: diagrama de clases y objetos Dinámico: diagramas de interacción, de estado y de actividades.
Vista de Procesos	Hilos y procesos que forman los mecanismos de sincronización y concurrencia del sistema.	Los aspectos estáticos y dinámicos se capturan con los mismos diagramas anteriores, pero haciendo énfasis en las clases activas.
Vista de Implementación	Componentes y archivos que se utilizan para ensamblar y hacer disponible el sistema físico.	Estático: diagrama de componentes Dinámico: diagramas de interacción, de estado y de actividades.
Vista de despliegue	Nodos que conforman la topología de hardware.	Estático: diagramas de despliegue Dinámico: diagramas de interacción, de estado y de actividades.

UNIDAD 4: INGENIERÍA DE REQUERIMIENTOS

¿Qué es un requerimiento?

"Característica del sistema, o una descripción de **algo que el sistema es capaz de hacer** con el objetivo de **satisfacer el propósito** del mismo"

¿Qué características tiene?

Necesario: Su omisión provoca que el sistema tenga una deficiencia.

Conciso: Es fácil de leer y entender.

Completo: No se necesitan ampliar detalles para su comprensión.

Consistente: No se contradice con otro requerimiento.

No ambiguo: Solo se interpreta de una forma.

Verificable: Se puede hacer uso de distintos métodos de verificación.

¿Cuáles son sus niveles de descripción?

Requerimientos del usuario

- ✓ Se documentan los **deseos y necesidades del cliente** y se expresan en un lenguaje claro.
- ✓ La audiencia primaria es la comunidad del cliente, y la secundaria es la comunidad del desarrollador.

Requerimientos del sistema

- ✓ Se establecen de manera **detallada y estructurada** los **servicios y restricciones** del sistema.
- ✓ El documento de requerimientos, también llamado **especificación natural**, tiene que ser preciso, además de servir como contrato entre el comprador del sistema y los desarrolladores.
- ✓ La audiencia primaria es la comunidad del desarrollador, y la secundaria es la comunidad del cliente.

¿Cómo se relacionan estos requerimientos?

Se relacionan a través del documento de ERS, como se puede ver a continuación:



¿Cómo se clasifican los requerimientos?

Funcionales: descripción de lo que el sistema hace. ¿Qué información debe ser mantenida? ¿Qué necesita ser procesado?

Describen **como se comportará el sistema ante determinado estímulo**, es decir, se detallan las **transformaciones que el sistema realiza sobre las entradas para producir salidas**.

No funcionales: restricciones del ambiente donde está construido el sistema, además de restricciones globales sobre cómo tiene que ser construido y cómo debe funcionar el sistema. ¿Por qué fue creado el sistema?

Un ejemplo de limitantes del sistema es, el **rendimiento** (en tiempo y espacio), la **fiabilidad**, el **mantenimiento**, **seguridad**, **portabilidad**, etc.

¿Cómo se clasifican los requerimientos no funcionales?



REQUERIMIENTOS	DESCRIPCIÓN
Requerimientos del Producto	Especifican el comportamiento del producto (rapidez de ejecución, tamaño en memoria y disco requerido, fiabilidad, seguridad, etc.)
Requerimientos de la Organización	Proviene de las políticas existentes en la organización del cliente y del desarrollador (cómo se utilizará el sistema, qué lenguaje de programación, estándares del entorno)
Requerimientos externos	Derivan de factores externos al sistema (entes reguladores, legislativos y éticos)

¿Qué es la Ingeniería de Requerimientos?

Es el proceso de descubrir, analizar, documentar y verificar los **servicios y restricciones** que tiene que **considerar el sistema** a desarrollar.

Esto permite una especificación de los requerimientos **completa, consistente y no ambigua**, donde se describirán las **funciones que realizará el sistema**.

¿Cuáles son sus actividades?



ACTIVIDAD	CONCEPTO	RESULTADO	TÉCNICAS
Elicitación	Adquirir conocimiento del trabajo del cliente, buscando comprender sus necesidades y detallando restricciones medioambientales.	Conjunto de requerimientos de todas las partes involucradas.	Entrevistas, cuestionarios, observaciones, JAD.
Especificación	Describir el requerimiento.	Forma de contrato entre usuarios y desarrolladores, el cual define el comportamiento funcional deseado del software.	Casos de Uso, complementado con herramientas gráficas.
Validación	Parte de los anteriores y realiza la integración y validación final de lo obtenido.	Modelo de requerimientos en línea con las expectativas de los clientes.	Revisiones de requerimientos, prototipos, casos de prueba.

UNIDAD 5: ELICITACIÓN DE REQUERIMIENTOS

La elicitación es el proceso de descubrir, analizar, documentar y verificar los servicios y restricciones que debe considerar el sistema a desarrollar.

Es donde se va a adquirir el conocimiento de como se realiza el trabajo y qué acciones hace el cliente y futuro usuario, para poder así comprender sus necesidades y también encontrar restricciones medio ambientales.

El objetivo de todo esto es lograr un conocimiento del problema para poder producir una especificación rigurosa del software necesario para resolver dicho problema.

Actividades → conformadas por las siguientes tres acciones:

1. Identificar las **fuentes** del conocimiento: todo aquello que va a proveer de información o que contiene la información necesaria.
2. Adquirir conocimiento a través de técnicas de **elicitación**.
3. Decidir sobre la **relevancia** del conocimiento: ordenar requerimientos en un orden lógico, para poder clasificarlos según sean funcionales o no, y también para priorizar el orden en el que se van a abarcar.

Interacción con otros procesos

Al descubrir los requerimientos del usuario, conoceremos todo el contexto y situación real que van a proveer de toda la información que se tiene que **especificar**

En ese proceso de feedback con el cliente, se pueden llegar a encontrar aun mas requerimientos, es por eso que se tiene que rehacer la **elicitación** y otra nueva **especificación** con los nuevos conocimientos, además de modificar lo que resultó de la **validación** de los ya especificados.

El resultado es el **producto**: el conjunto o listado de requerimientos establecidos por todas las partes involucradas, de lo que pedía el cliente y de las mejoras que se van a implementar (traducir problema en necesidad)



TECNICAS DE ELICITACIÓN

Volviendo al segundo paso de las actividades, se describen entonces las **técnicas de elicitación**:

Técnica	Descripción	Metodología
ENTREVISTAS	Conversación dirigida con un propósito específico (opiniones, sentimientos, objetivos, etc.) que se basa en un formato de preguntas y respuestas	1- <u>Planear la entrevista</u> : conocer bien al entrevistado y sus características 2- <u>Realizar la entrevista</u> : crear un clima cómodo de confianza para que se pueda transmitir la información correctamente. 3- <u>Analizar la información obtenida y elaborar un informe</u>
	Características extras	Técnicas
	Se pueden realizar tanto preguntas abiertas como cerradas, hay que tener en cuenta que ambas tienen sus ventajas y desventajas: Abiertas Baja Bajo Baja Mucha Mucha Difícil Confiabilidad de los datos Uso eficiente del tiempo Precisión de los datos Amplitud y profundidad Habilidad requerida del entrevistador Facilidad de análisis Cerradas Alta Alto Alta Poca Poca Fácil Una entrevista puede estar estructurada (donde todo está planeado de antemano) o ser no estructurada (estructura no formal, pero no significa que no haya preparación previa)	<u>Estructura piramidal</u> (organización inductiva): comenzar con preguntas detalladas y luego ir ampliando los tópicos haciendo preguntas abiertas <u>Estructura en embudo</u> (organización deductiva): ir en sentido contrario a la piramidal <u>Estructura en diamante</u>: comenzar con preguntas cerradas, a la mitad de la entrevista pedir la opinión al entrevistado sobre los tópicos generales, y finalmente replantear las preguntas con mayor precisión. <u>Grabación</u> <u>Toma de notas</u>

Técnica	Descripción	Metodología																					
CUESTIONARIOS	Formato de preguntas que se responden por escrito. Sirven como suplemento de utilidad para las entrevistas. Se buscan revelar actitudes, conductas, opiniones y características de la otra persona.	Definir el objetivo: para qué Elaborar preguntas del cuestionario Seleccionar a las personas que lo realizarán																					
	Características extras																						
	Sirven para conseguir respuestas a preguntas específicas cerradas, las cuales luego pueden cuantificarse. Las preguntas tienen que ser completamente transparentes																						
	Inconvenientes de las preguntas abiertas y cerradas <table><thead><tr><th>Abiertas</th><th></th><th>Cerradas</th></tr></thead><tbody><tr><td>Lenta</td><td>Velocidad de conclusión</td><td>Rápida</td></tr><tr><td>Alta</td><td>Naturaleza explorativa</td><td>Poca</td></tr><tr><td>Alta</td><td>Amplitud y profundidad</td><td>Poca</td></tr><tr><td>Fácil</td><td>Facilidad de Preparación</td><td>Difícil</td></tr><tr><td>Difícil</td><td>Facilidad de análisis</td><td>Fácil</td></tr></tbody></table> Uso de escalas <table><tbody><tr><td> NOMINAL - Se utilizan para clasificar objetos. SI / NO V / F </td><td> ORDINAL - Además de realizar una clasificación implican un arreglo por categorías. M / R / B / MB / E </td></tr><tr><td> DE INTERVALO - Tienen la característica que la diferencia que existe es la misma entre los intervalos de cada uno de los números. 1 - 2 - 3 ... - 10 </td><td> PROPORCIONAL - Son similares a las escalas de intervalos pero incluyen el cero. 0 .. 2 .. 4 .. 6 .. 8 </td></tr></tbody></table>		Abiertas		Cerradas	Lenta	Velocidad de conclusión	Rápida	Alta	Naturaleza explorativa	Poca	Alta	Amplitud y profundidad	Poca	Fácil	Facilidad de Preparación	Difícil	Difícil	Facilidad de análisis	Fácil	NOMINAL Se utilizan para clasificar objetos. SI / NO V / F	ORDINAL Además de realizar una clasificación implican un arreglo por categorías. M / R / B / MB / E	DE INTERVALO Tienen la característica que la diferencia que existe es la misma entre los intervalos de cada uno de los números. 1 - 2 - 3 ... - 10
Abiertas		Cerradas																					
Lenta	Velocidad de conclusión	Rápida																					
Alta	Naturaleza explorativa	Poca																					
Alta	Amplitud y profundidad	Poca																					
Fácil	Facilidad de Preparación	Difícil																					
Difícil	Facilidad de análisis	Fácil																					
NOMINAL Se utilizan para clasificar objetos. SI / NO V / F	ORDINAL Además de realizar una clasificación implican un arreglo por categorías. M / R / B / MB / E																						
DE INTERVALO Tienen la característica que la diferencia que existe es la misma entre los intervalos de cada uno de los números. 1 - 2 - 3 ... - 10	PROPORCIONAL Son similares a las escalas de intervalos pero incluyen el cero. 0 .. 2 .. 4 .. 6 .. 8																						

Técnica	Descripción
ANÁLISIS DE DOCUMENTACIÓN	Se busca comprender el papel de los documentos en una organización. La información buscada son hechos, cifras, información financiera, el contexto organizacional, etc. Los documentos pueden ser cuantitativos o cualitativos. Si son cuantitativos: reportes para toma de decisiones, reportes de desempeño y registros de transacciones. Se toman solo los modelos que sirve para cada tipo de documento. Sirven para aprender cómo se utilizan. Si son cualitativos: información sin formalidad, sitios web, manuales. Sirven para dar información sobre quién hace una tarea, los procedimientos establecidos y la forma en que se comunican.
OBSERVACIÓN	Se busca profundizar en lo que se hace realmente, y no solo en lo que se dice o se tiene documentado. Se buscan actividades, mensajes, relaciones e influencias. Otras razones por las que se hace observación, es para confirmar lo que las entrevistas y cuestionarios han detectado, o para rebatir o anular lo que otros métodos encontraron.
JAD (JOINT APPLICATION DESIGN – DISEÑO CONJUNTO DE APLICACIONES)	Sirve para promover la cooperación, el trabajo en grupo entre compradores, usuarios y desarrolladores. Se realiza cuando los usuarios quieren una solución fuera de lo común, o cuando los analistas pronostican que se obtendrán mas ideas si se colabora grupalmente. La sesión JAD consta de 8 a 12 usuarios, 1 o 2 observadores analistas, 1 escriba que formalice todo lo que se haga, un líder de sesión, un analista en sistemas y un patrocinador. La sesión se realiza fuera de la organización, dura entre 2 a 4 días y se asegura la participación de todos los involucrados. Consta de 3 fases: Customization (organización de grupo y materiales), Session (uno o más encuentros que involucran a desarrolladores y usuarios, se comienzan a visualizar las ideas y vistas sobre el sistema) y Wrap-up (conversión de la información de la Session en su forma final → puede ser un documento de especificación de requerimientos)

TORBELLINO DE IDEAS	Es una técnica de grupo que permite a las personas sugerir y explorar ideas en una atmosfera libre de criticas y prejuicios. Participan alrededor de 4-10 personas Un líder únicamente da comienzo a la sesión, no significa que tenga más poder. Consta de dos fases: Generación: Se les pide a los participantes que digan tantas ideas como sea posible, sin discutir la importancia. Consolidación: Se discute cuáles de esas ideas tienen mas sentido y se organizan en esenciales, no esenciales y apropiadas para una segunda versión del sistema. Finalmente se guarda un registro de las ideas que quedaron, con sus prioridades y comentarios relevantes de la fase.
----------------------------	--

UNIDAD 6: ESPECIFICACIÓN DE REQUERIMIENTOS

En la actividad de especificación de requerimientos, se traduce la información recopilada en la actividad Elicitación, en un documento que define un conjunto de requerimientos, el cual a su vez nos da la idea de como será el comportamiento deseado del sistema.

Este documento se llama **ERS** (Especificación de **R**equisitos de **S**oftware), y se define como la documentación de requisitos esenciales del software y de sus interfaces externas. Puede incluir:

Requerimientos del usuario: requerimientos del cliente y usuario final del sistema.

Requerimientos del sistema: descripción detallada de funcionalidad a proporcionar.

¿Cuáles son sus propósitos?

- ✓ **Comunicación entre clientes, usuarios, analistas**: mayor convicción con el cliente y definir el comportamiento externo del sistema.
- ✓ **Soporte a las actividades de prueba del sistema**: estimular al sistema con escenarios de prueba representativos para mostrar que el sistema construido SI cumple con los requerimientos.
- ✓ **Control de evolución del sistema de software.**

¿Qué se incluye en una ERS?

1. Una descripción concisa y detallada de la interfaz externa del sistema con su entorno, incluyendo otro software, puertos de comunicaciones, hardware y usuarios humanos.
2. Requerimientos funcionales y no funcionales.

¿Qué NO se incluye en una ERS?

1. Requerimientos del proyecto (costos, fases)
2. Diseños (solo van las restricciones del diseño)
3. Planes de aseguramiento del producto (planes de pruebas, o de calidad)

¿Quiénes son los usuarios de una ERS?

USUARIO	DESCRIPCIÓN
Clientes del sistema	Son los que definen los requerimientos, a la vez que especifican cambios en los mismos.
Administradores	Son los que planifican una oferta por el sistema y planifican el proceso de desarrollo del sistema.
Ingenieros de Sistemas	Comprenden qué sistema debe desarrollarse.
Ingenieros probadores del sistema	Desarrollan pruebas de validación.
Ingenieros mantenedores del sistema	Comprenden el sistema y las relaciones entre sus partes.

¿Qué características tiene una buena ERS?

Una buena ERS es:

- ✓ Correcta
- ✓ No ambigua
- ✓ Completa
- ✓ Verificable
- ✓ Consistente
- ✓ Modificable
- ✓ Rastreadable
- ✓ Clasificada (importancia y estabilidad)

¿Qué técnicas de Especificación se pueden utilizar?

Función: ¿qué hace el sistema?

Información: ¿qué información es utilizada por el sistema?

Tiempo: ¿cuándo sucede algo en el sistema?

Estas tres técnicas pueden utilizarse por separado o se puede hacer uso de una combinación de las mismas.

¿Cómo es la estructura de una ERS?

Si bien no es obligatorio este esquema, cualquier ERS bien escrita debería de incluir toda la información de a continuación:

INTRODUCCIÓN	Objetivo: propósito de la ERS y a quién se dirige.
	Ámbito: nombre del sistema, explicación de lo que hará y que no hará el sistema, descripción de metas que se esperan alcanzar.
	Definiciones, siglas y abreviaturas: se definirán todos los términos y abreviaciones utilizados en la ERS.
	Referencias: lista completa de todos los documentos referenciados.
	Visión Global: descripción del contenido del resto de la ERS y cómo estará organizada.
DESCRIPCIÓN GENERAL	Perspectiva del producto: se relaciona el futuro producto con otros productos, se debe definir si además el producto es independiente de otros productos, o si es parte de un sistema mayor.
	Funciones del producto: resumen de las funciones del futuro sistema, sin especificar.
	Características del usuario: características generales de los usuarios del producto, incluyendo sus restricciones.
	Limitaciones generales: políticas de la empresa, limitaciones de hardware, requisitos de habilidad, consideraciones acerca de la seguridad.
	Supuestos y dependencias: son factores que, si cambian, pueden afectar a los requisitos.
REQUERIMIENTOS ESPECÍFICOS	Requisitos funcionales: se detallan uno por uno los requerimientos funcionales, cada uno con su introducción, entrada, procesamiento y salidas.
	Requisitos de Interfaz Externa: interfaces de usuario, hardware, software.
	Restricciones de diseño: todo aquello que restrinja a las decisiones relativas al diseño de la aplicación, tal como estándares o limitaciones de hardware.
	Atributos de calidad: seguridad y mantenimiento, se especifican qué usuarios están autorizados o no a ciertas tareas.
	Restricciones de rendimiento: carga que se espera que tenga que soportar el sistema, por ejemplo, número de terminales, número esperado de usuarios simultáneamente conectados.
	Otros requisitos: cualquier otro requisito que no haya encajado en las otras secciones.
APÉNDICES	Índice

DESCUBRIMIENTO DE REQUISITOS

- 1- Se elabora una Lista de Características: sirve para la aparición de ideas. Una característica puede ser un requerimiento funcional global o detallado, o un requerimiento no funcional.
- 2- Comprender el contexto del sistema (Modelo de Dominio)
- 3- Capturar requisitos funcionales: se puede hacer mediante Casos de Uso. Estos capturan tanto los requisitos funcionales como los no funcionales de cada Caso de Uso.
Para un usuario, un Caso de Uso es un modo de utilizar el sistema.
- 4- Capturar requisitos no funcionales: especifican propiedades del sistema como restricciones del entorno o de la implementación, que limitan la elección de alternativas en la etapa de diseño y construcción del sistema. Algunos requisitos no funcionales se asocian a determinados casos de uso, y otros se asocian a un caso de uso concreto o una clase del mundo real.

ESPECIFICACIÓN DE REQUERIMIENTOS CON CASOS DE USO

Un caso de uso describe los comportamientos de un sistema, bajo ciertas condiciones en las que el sistema tiene que responder ante los requerimientos de los involucrados.

Entonces, un caso de uso es una porción de funcionalidad que el sistema ofrece para aportar un resultado de valor para sus actores.

- ✓ Representa cada forma en que los actores usan el sistema.
- ✓ Especifica una secuencia de acciones que el sistema puede llevar a cabo interactuando con sus actores.
- ✓ Se deben describir brevemente con algunas frases que resumen las acciones.

Nomenclatura: Verbo en infinitivo + objeto

Simbología:



ACTOR

Un actor es el rol de la persona, hardware, o incluso otro sistema, que interactúa con el sistema en un caso de uso. Al asociar un actor con un caso de uso, además se debe hacer una descripción del rol.

Rol: cada usuario puede tener uno o mas roles al interactuar con el sistema. Cada rol se identifica como un actor. Ej.: empleado de atención al público, cajero, reloj de sistema, sensor de temperatura.

Actor primario: es el que tiene un objetivo claro que tiene que ser tenido en cuenta y concretado con la ayuda del sistema de información. Ej.: encargado cuentas automotor → registrar baja de vehículo.

Actor secundario: es de quien el sistema necesita ayuda o un determinado comportamiento para que se pueda cumplir el objetivo del actor primario. Ej.: director de rentas (autorizar baja de vehículo del actor primario)

¿Cómo encontrar actores?

Hay que hacerse las siguientes preguntas:

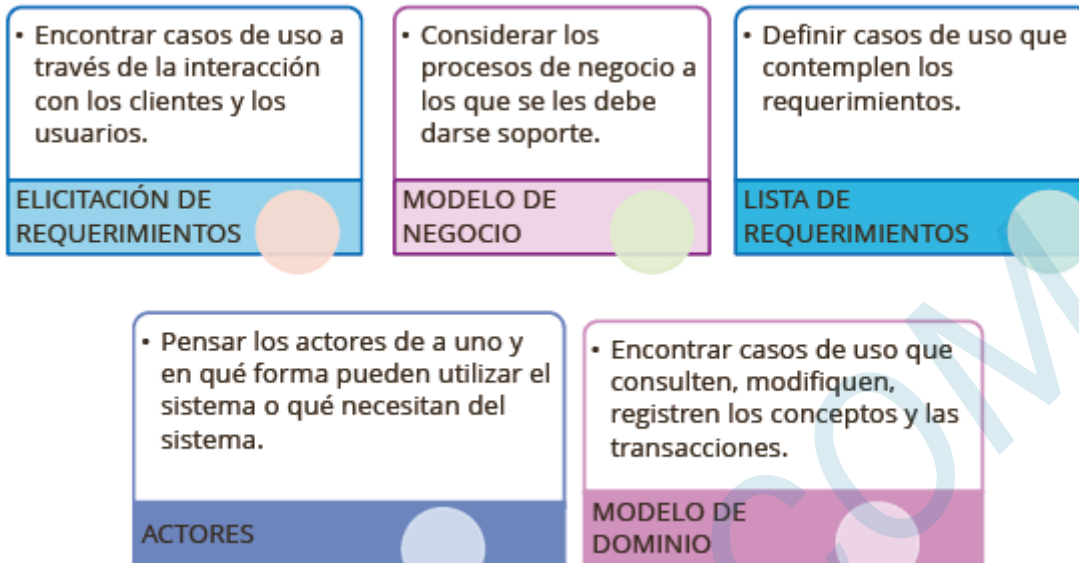
- ¿Quién va a proveer, usar o quitar información?
- ¿Quién hará esta funcionalidad?
- ¿Quién está interesado en este requerimiento?
- ¿En que parte de la organización será usado el sistema?
- ¿Quién dará soporte y mantenimiento al sistema?
- ¿Cuáles son los recursos externos del sistema?
- ¿Qué otros sistemas necesitaran interactuar con este sistema?

Especialización

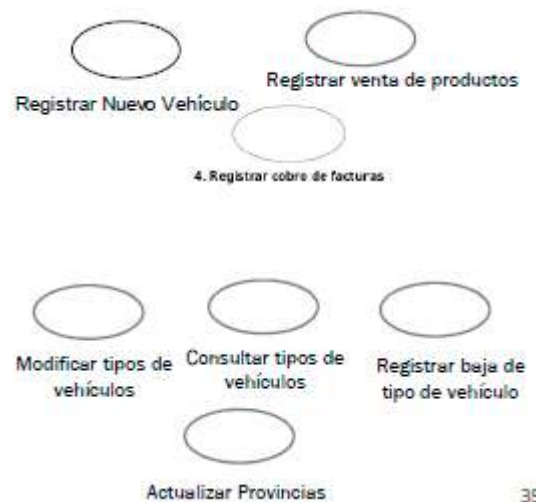
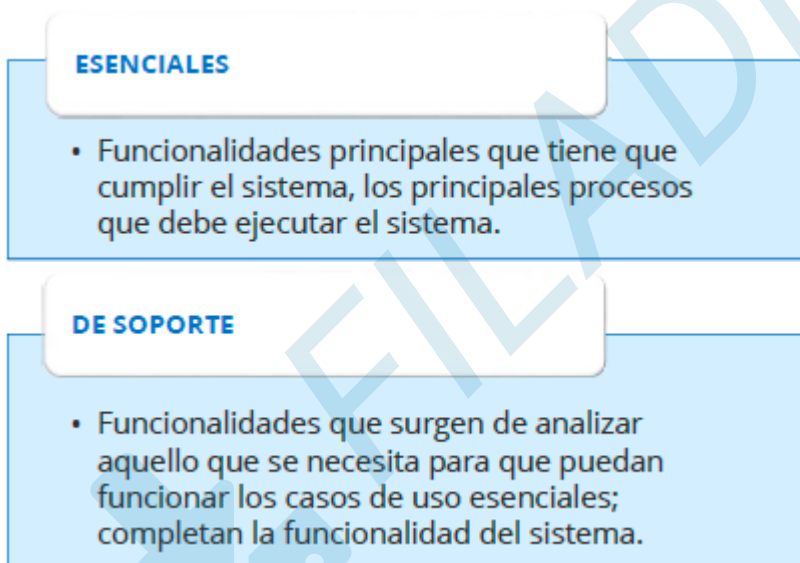
Los actores especializados (hijos) heredan el comportamiento del actor padre.



Fuentes:



Categorías



Un caso de uso puede ser:

Concreto: es iniciado por un actor.

Abstracto: no es iniciado nunca por un actor, únicamente es instanciado por otro caso de uso.

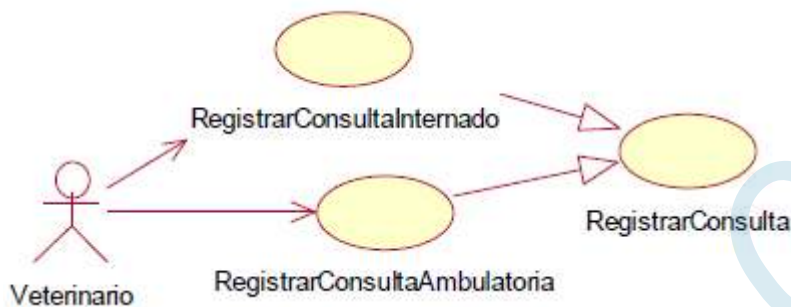


¿Cómo estructurar el Modelo de Casos de Uso?

Implica determinar relaciones entre los casos de uso:

Generalización entre Casos de Uso:

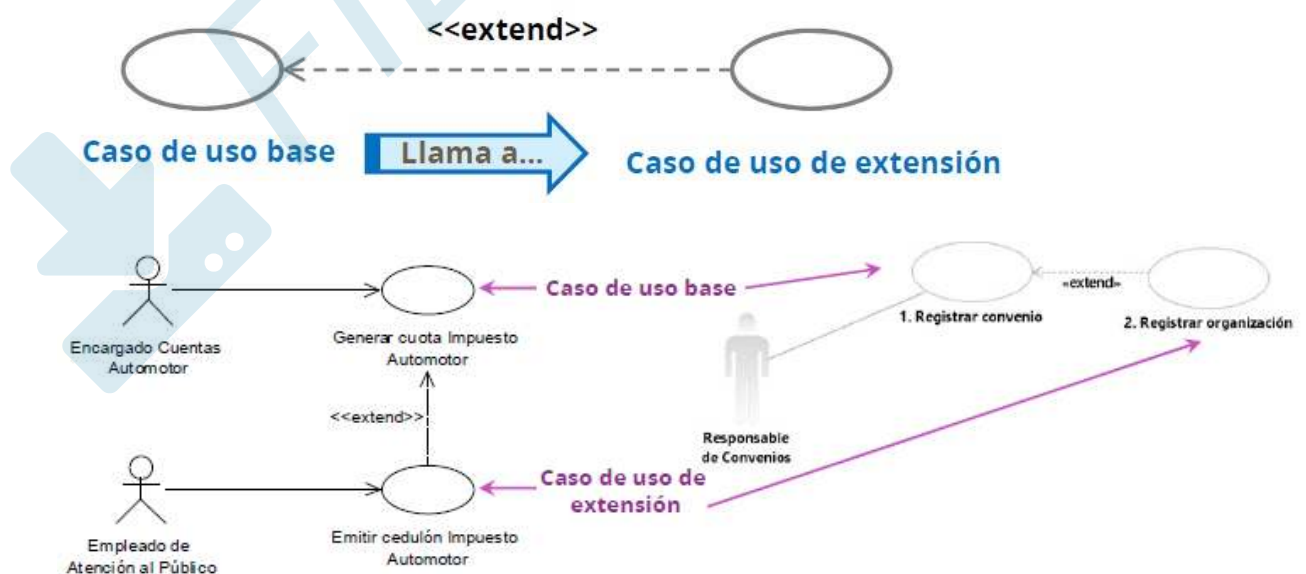
El hijo hereda comportamientos del caso de uso padre. Puede agregar y/o modificar comportamiento a lo heredado. Ejemplo:



Extensión entre Casos de Uso:

Un caso de uso base incorpora implícitamente el comportamiento de otro caso de uso de uso.

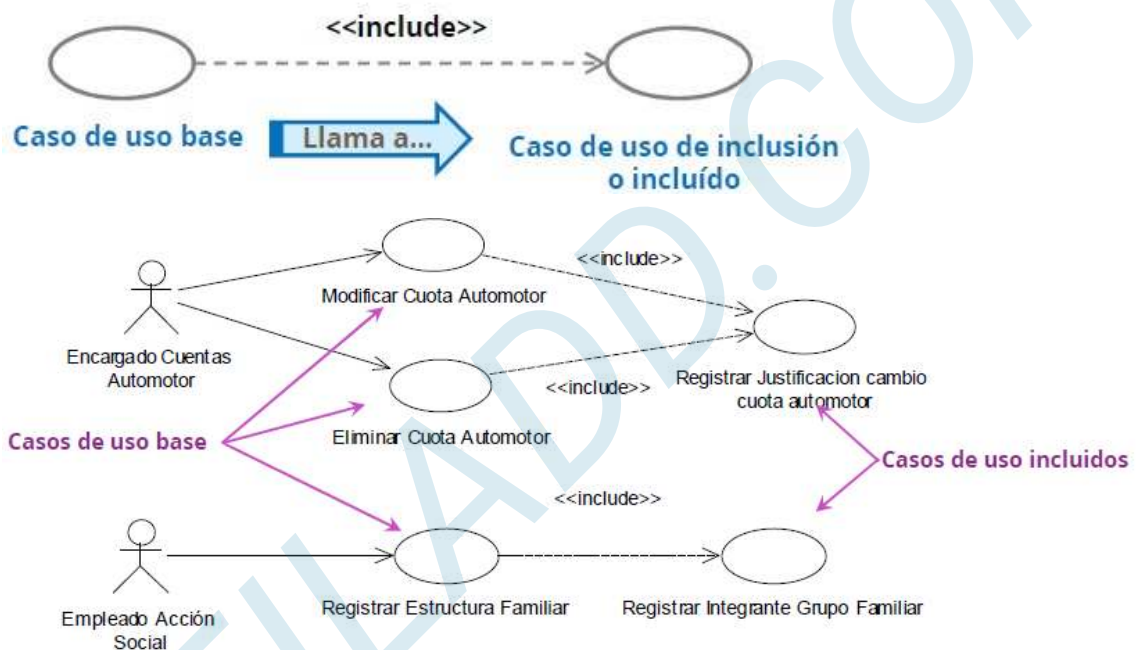
- ✓ Algunas veces puede ser instanciado directamente por un actor.
- ✓ Puede instanciarse para extender el comportamiento de un caso de uso base.



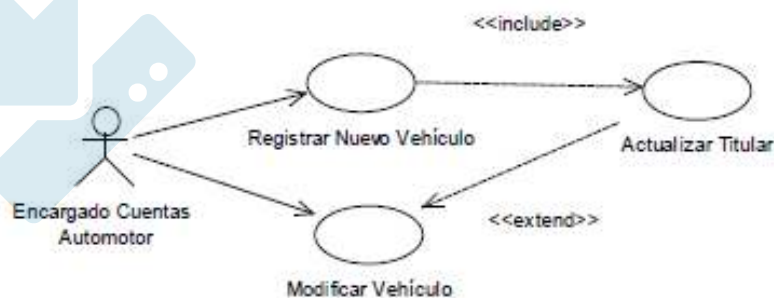
Inclusión entre Casos de Uso

Un caso de uso incorpora explícitamente el comportamiento de otro caso de uso en el lugar especificado en el caso base.

- ✓ Puede ser instanciado por un actor si constituye un curso de acción completo en sí mismo.
- ✓ Se usa para abstraer el comportamiento común entre varios casos de uso.
- ✓ También se usa para abstraer un comportamiento complejo que complica el caso de uso base.



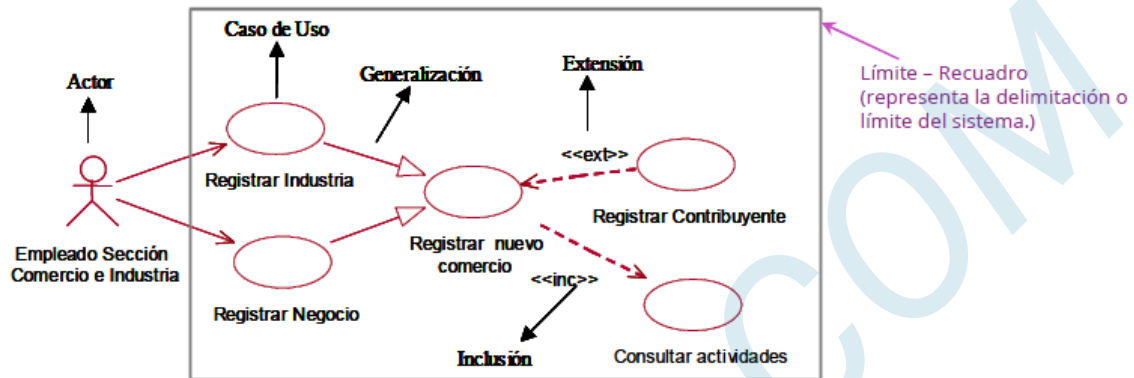
Ejemplo extensión/inclusión



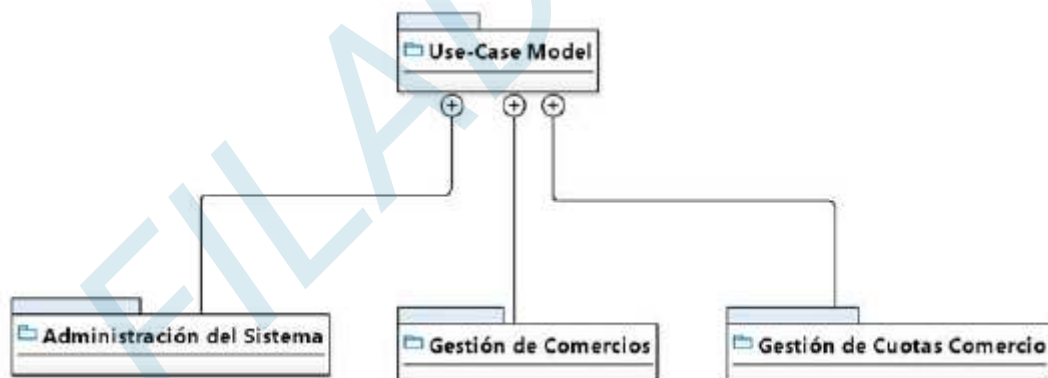
Diagramas de Casos de Uso

- ❖ Modelan el comportamiento de un sistema o subsistema.
- ❖ Consiste en elaborar diagramas y descripciones para explicar el modelo de casos de uso en totalidad.

Ejemplo:



- ❖ Puede a su vez organizarse en conjuntos de casos de uso, formando paquetes de casos de uso. Ejemplo:



Escenarios y pasos: son los eventos que impiden o permiten a un actor a alcanzar su meta.

Un escenario finaliza en un éxito o un fracaso. Puede representar:

Flujo normal o principal: es el que cumple la meta u objetivo del caso del caso de uso.

Flujo alternativo: pueden ser exitosos o una condición de error (fracaso).

Cada paso hace avanzar al usuario hacia su meta. Puede contener condiciones alternativas y de error.

La descripción de un caso de uso, debería incluir:

Al principio...

- El estado inicial, como **precondición**.
- **Cómo y cuándo comienza** el caso de uso.

El desarrollo...

- El **orden** requerido en el que las **acciones** se deben ejecutar
- Las descripciones de **caminos alternativos** que están **incluidos** junto con la descripción del camino básico.
- Las descripciones de **caminos alternativos** que han sido **extraídos** de la descripción del camino básico.
- La **interacción del sistema con los actores** y qué cambios producen.
- La utilización de **objetos, valores y recursos del sistema**.
- La descripción explícita de **lo que hace el sistema**, separando las responsabilidades del sistema de las **acciones de los actores**.

El fin...

- **Cómo y cuándo terminan** los casos de uso.
- Los posibles estados finales, como **postcondiciones**. Una postcondición contempla los resultados posibles de un caso de uso.

Patrones de Casos de Uso

Un patrón describe un problema que ocurre una y otra vez en nuestro entorno, la solución a ese problema se puede aplicar un millón de veces, sin hacer lo mismo dos veces.

"Ante este problema, lo mejor es aplicar tal solución"

Los patrones son una herramienta de diagnóstico y deberían ser usados para revisar los Casos de Uso definidos con el objetivo de perfeccionar y de construir un diagrama de Casos de Uso que tenga estándares de calidad.

Categorías de patrones de Casos de Uso

Patrones estructurales: describen componentes básicos, cómo tendrían que estar organizados, ofreciendo criterios que fundamenten su uso.

Patrones de desarrollo: describen características que tienen que ver con el trabajo en equipo y los estilos de escritura, para plantear mejoras al momento de redactar.

Patrones estructurales



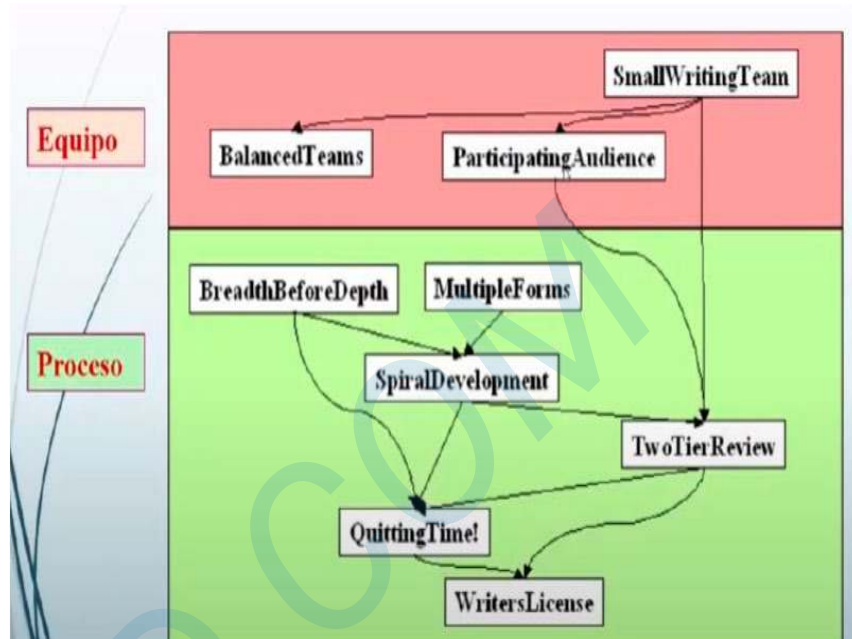
Conjunto de Casos de Uso	1. SharedClearVision	Preparar declaración de propósitos que describa los objetivos del sistema de información, y distribuirlo a todos los involucrados con el proyecto.
	2. VisibleBoundary	Tener en claro el límite entre el sistema y el entorno, y enumerar el personal que interactúan con el sistema (Actores del sistema)
	3. ClearCastOfCharacters	Identificar actores con los cuales el sistema tiene que interactuar y el papel que juega cada uno (Rol de cada actor)
	4. UserValuedTransactions	Identificar servicios (req. Func.) de valor que el sistema entrega a los actores para que puedan satisfacer sus propósitos.
	5. EverUnfoldingStory	A medida que se construye el diagrama de C.U, convendrá organizar los casos por temas o paquetes, aplicando algún criterio.
Casos de Uso	6. CompleteSingleGoal	Definir un objetivo para cada caso de uso
	7. VerbPhraseName	Definir el nombre del caso de uso
	8. ScenarioPlusFragments	Realizar descripción del caso de uso
	9. ExhaustiveAlternatives	Capturar todas las alternativas y fallas que deben manejarse en el caso de uso
	10. Adornments	Campos adicionales fuera de la plantilla que contengan información suplementaria útil.
	11. PreciseAndReadable	Escribir el caso de uso para que sea legible y preciso .
Escenarios	12. DetectableConditions	Incluir solamente condiciones detectables y perceptibles (que aporten utilidad/descripción)
	13. LeveledSteps	Mantener escenarios con una cantidad de pasos nivelados (mantener a todos en el mismo nivel)
Pasos	14. ActorIntentAccomplished	Distinguir y escribir en diferentes pasos las actividades del actor y del sistema
	15. ForwardProgress	Eliminar o unir pasos que no hagan avanzar al actor hacia la meta (simplificar pasos)
	16. TechnologyNeutral	Escribir cada paso en una manera tal que la tecnología no este involucrada.
Relaciones	17. CommonSubBehavior	Estructurar el modelo de casos de usos con relaciones de inclusión
	18. InterruptionsAsExtensions	Cuando un curso de acción alternativo interrumpe un numero de pasos en un escenario, crear un caso de uso de extensión
	19. PromotedAlternative	Mover alternativas complejas a un caso de uso separado.
	20. CaptureTheAbstraction	Considerar crear un caso de uso abstracto generalizado y colocar en casos de usos "hijos" cada escenario distinto que especializa esa abstracción.

Patrones de desarrollo

Categorías de patrones de desarrollo:

- Team (Equipo)
- Process (Proceso)

Los patrones de equipo y proceso describen características de practicas de escritura en casos de uso probadas y ofrecen criterios para medir la calidad del proceso de escritura.



- Editing (Revisión)

Equipo	21. SmallWritingTeam	Trabajar en el refinamiento de un producto en un grupo de 2 a 3 personas.
	22. ParticipatingAudience	Involucrar activamente a los clientes y usuarios en el proceso de desarrollo de casos de uso.
	23. BalancedTeam	Proveer al equipo con personal de diferentes especialidades.
Proceso	24. BreadthBeforeDepth	Desarrollar primero una vista general de los casos de uso y progresivamente agregar detalles.
	25. SpiralDevelopment	Desarrollar casos de usos de una manera iterativa, progresivamente mejorar la precisión y exactitud.
	26. MultipleForms	Seleccionar formato en base a riesgos asociados con el proyecto y las preferencias de las personas involucradas.
	27. TwoTierReview	Hacer dos revisiones, la primera con un grupo pequeño, varias veces, y la segunda con un grupo grande 1 o 2 veces.
	28. QuittingTime	Detener desarrollo de casos de uso que estén completos
	29. WritersLicense	El escritor puede reclamar "licencia de escritor" en su estilo de escritura.
Revisión	30. RedistributeTheWealth	Mover un pasaje largo, difícil de manejar, a otro caso de uso
	31. MergeDroplets	Unir casos de usos diminutos o fragmentos de C.U en un caso de uso con objetivos emparentados.
	32. CleanHouse	Remover C.U que no agregan valor al sistema.

Algunos ejemplos de patrones:

Objetivo del SI: Patrón #1. Compartir una visión clara

Diagramas de Casos de Uso

Patrón #2. Límite Visible

Patrón #3. Reparto claro de roles

Nombre del Actor	Descripción del rol del actor
Administrador de Sistema	Encargado de administrar el perfil de los usuarios y permisos a cada usuario.
Encargado de Asociaciones	Encargado de gestionar las asociaciones de las personas y mantener actualizados los datos personales del responsable de la generación de los informes.

Patrón #4. Transacciones de valor para el usuario

Patrón #7. Nombrar con una frase verbal

Patrón #6. Completar un objetivo simple

Relaciones entre CU:

Patrón #17. Comportamiento común subordinado

Patrón #18. Interrumpir con extensiones

Patrón #19. Promover alternativa

Patrón #20. Abstracción capturada

Paquetes de CU: **Patrón #5. Siempre contar una historia**

29

EJEMPLO: CLÍNICA PRIVADA: CU 1. REGISTRAR PACIENTE

Curso Normal

1. El Responsable de Recepción de Pacientes (RRP) ingresa a la opción para registrar un nuevo paciente.
2. El sistema muestra los tipos de documento y solicita se seleccione el tipo de documento y se ingrese el número de documento del paciente.
3. El RRP selecciona el tipo de documento e ingresa el número de documento.
4. El sistema valida si existe un paciente registrado con ese tipo y número de documento, y no existe.
5. El RRP ingresa los siguientes datos personales del paciente: nombre y apellido, fecha de nacimiento, teléfono, dirección de e-mail.
6. El sistema calcula y muestra la edad del paciente. (Ver observación 1)

Patrón #14. Actor intenta cumplir

P#8. Escenarios más Fragmentos

P#9. Alternativas Exhaustivas

P#11. Preciso y Legible

P#12. Condiciones Detectables

P#15. Progresar Hacia Delante

Cursos Alternativos

- A1: Paso 4. Ya existe un paciente registrado con ese documento.
El sistema muestra apellido, nombre y el número de historia clínica del paciente registrado.
El RRP desea ingresar otro documento.

Post-Condiciones:

Éxito:

Se registró un nuevo paciente y se generó la historia clínica.

Fracaso:

El CU se cancela cuando:

- El paciente ya está registrado y el RRP no desea ingresar un nuevo documento (A1)
Mensaje: "El paciente ya está registrado"

31

Patrón #16. Tecnología Neutral

Ejemplo: Clínica

Así, es correcto describir la acción independientemente del elemento o recurso del prototipo de interfaz.

14. El sistema solicita se confirme la registración del paciente.
15. El RRP confirma la registración del paciente.
16. El sistema genera un número de historia clínica correlativo para el paciente y lo muestra.

No es correcto describir con la mención del nombre del botón.

The screenshot shows a patient registration form with fields for Apellido, Nombre, Fecha de nacimiento, Estado (Año), Teléfono, Email, and a section for Domicilio (Calle, Número, Ciudad, Código Postal, N° Aliado). At the bottom, there are three buttons: 'Aceptar' (blue), 'Cancelar' (grey), and 'Salir' (yellow). A red box highlights the 'Aceptar' button, and a red arrow points from the text 'No es correcto describir con la mención del nombre del botón.' to it.

EJEMPLO: CLINICA

Curso Normal

4. El sistema valida si existe un paciente registrado con ese tipo y número de documento, y no existe.
5. El RRP ingresa los siguientes datos personales del paciente: nombre y apellido, fecha de nacimiento, teléfono, dirección de e-mail.

Cursos Alternativos

A1: Paso 4. Ya existe un paciente registrado con ese documento.

El sistema muestra apellido, nombre y el número de historia clínica del paciente registrado.
El RRP desea ingresar otro documento.

Patrón #12. Condiciones Detectables

Curso Normal

12. El sistema solicita se seleccione el tipo de paciente (Particular o Con Obra Social).
13. El RRP selecciona paciente particular.

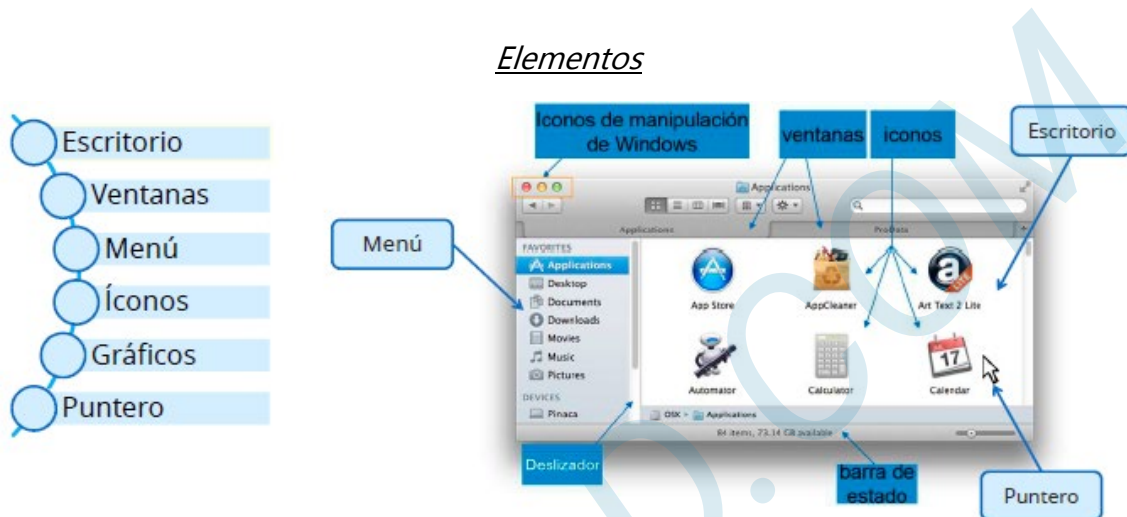
Cursos Alternativos

A4. Paso 13. El RRP selecciona paciente con obra social.

El sistema muestra las obras sociales para su selección.
El RRP selecciona una obra social e ingresa número de afiliado.

INTERFAZ DE USUARIO

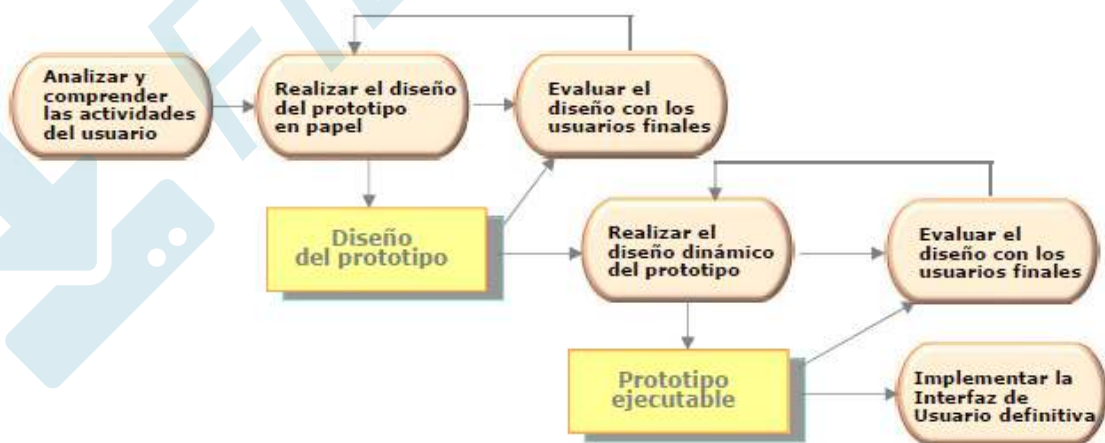
La interfaz de usuario (IU o GUI) es la que permite establecer la comunicación entre el usuario y la computadora. Debe estar bien diseñada para hacer que el usuario entienda el sistema que va a utilizar.



Ventajas

- ✓ Fáciles de aprender y utilizar
- ✓ Usuarios cuentan con múltiples ventanas

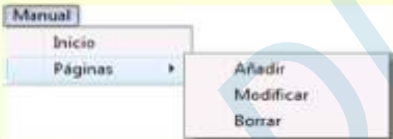
Proceso de diseño de la GUI



Cosas a tener en cuenta en el diseño de la GUI

Familiaridad del usuario	Usar terminos y conceptos que a los que los usuarios ya esten acostumbrados.
Consistencia	El usuario deberia saber como funciona un elemento solo con verlo.
Minima sorpresa	El comportamiento del sistema no debe provocar sorpresa a los usuarios.
Recuperabilidad	Incluir mecanismos para permitir a los usuarios recuperarse frente a errores, a su vez de proporcionar formas de "deshacer", "rehacer", etc.
Guia al usuario	Proporcionar retroalimentacion frente a errores que sirvan de ayuda al contexto.
Diversidad de usuarios	Proveer características apropiadas para cada tipo de usuario del sistema.

Tipos de interaccion en una GUI

Estilo de Interacción	Descripción	Ejemplos de Aplicación
Manipulación directa		- Videojuegos - Sistemas CAD
Selección de menús		- La mayoría de los sistemas de propósito general
Rellenado de formularios		- La mayoría de los sistemas de transaccionales
Lenguaje de comandos		- Sistemas operativos - Sistemas de comandos y control
Lenguaje Natural		- Sistemas de recuperación de información.

Formas de presentación de la información al usuario

Como texto	Se usa para presenten información numerica precisa, y la informacion cambia lentamente.	Uso de grillas, tablas, cuadros de texto.
Como gráfico	Se usa para presenten datos que cambian rapidamente. Se recomienda utilizar colores acordes para ayudar a los usuarios a la facil comprension. Los cambios de estado se representan con un cambio de color. Otro tip es que no se deben mezclar los colores azules y rojos ya que cansa la vista.	Uso de graficos de barra, curva, torta.

MÁQUINA DE ESTADOS

Es la especificación de la secuencia de estados por los que pasa un objeto a lo largo de su vida, en respuesta a los eventos, junto con las respuestas a los mismos.

"Toda clase puede tener una sola máquina de estado de comportamiento"

Diagrama de máquina de estados -> Muestra o visualiza una máquina de estados, enseñando el flujo de control entre estados dentro de un único objeto. Así se modela el comportamiento que está dirigido por sus eventos.

Los tres elementos básicos del diagrama son el **estado**, la **transición**, y los **eventos**.

Estado: conjunto de valores y atributos que tiene un objeto durante una cantidad finita de tiempo.

Su notación es:



Estado inicial: punto de comienzo por defecto para la máquina de estados o el subestado. Notación: ●

Estado final: indica que la ejecución de la máquina de estados o del estado que lo contiene ha finalizado. Notación: ●

Ambos son **pseudoestados** (estados especiales).

Un estado tiene:

- **Nombre** -> Se expresa en verbo conjugado (ado/ada), y la primera letra de cada palabra va en mayúscula.
 - Efectos de entrada/salida
 - Transiciones internas
- Subestados (estructura anidada de un estado que engloba otros estados)
 - Eventos diferidos
 - Actividades

Transición: Relación entre dos estados, indica que el objeto que esté en el primer estado va a hacer varias acciones, y luego va a entrar en el segundo estado cuando ocurra X evento específico y se satisfagan ciertas condiciones.



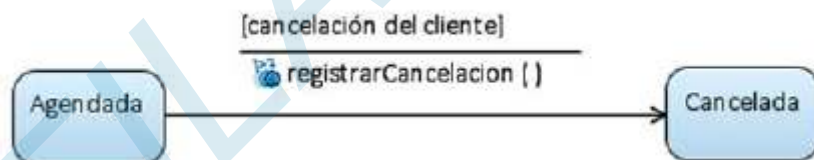
Una transición tiene:

- **Un estado origen**
- Un evento de disparo
- Una condición de guarda: una situación de validación.
- Un efecto (acción)
- **Un estado destino**

Su sintaxis es:

nombreEvento (parámetros opcionales) [condición de guarda opcional] / lista de acciones opcional

El nombre del evento se omite en las **transiciones de finalización**. Ejemplo:



Auto transición -> El estado de origen y el estado de destino es el mismo:



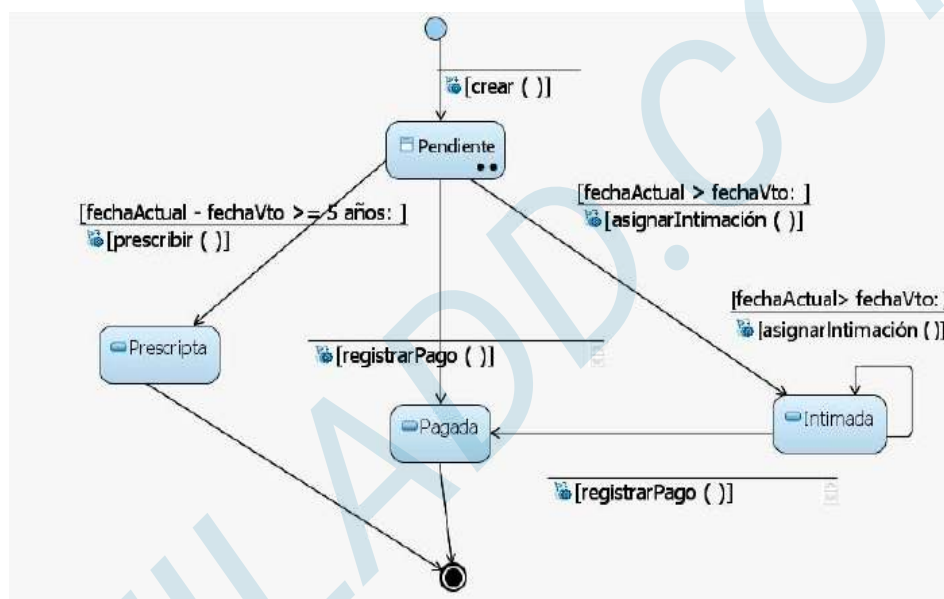
Evento: Estimulo que puede disparar una transición de estado.

Existen cuatro tipos de evento:

- **Evento de llamada**
 - Evento de señal
 - Evento de cambio
 - Evento de tiempo

Evento de llamada -> es cuando se pide que se haga una operación específica en una instancia de la clase.

Ejemplo integrador:



Relación de todo esto con otros modelos

Con el diagrama de clases	Los eventos de llamada que figuran en el diagrama de estados, tienen que estar presentes en la definición de la clase. A su vez, debe existir un atributo estado que será modelado mediante otra clase.
Con el diagrama de Casos de Uso	Debe existir al menos un Caso de Uso para cada método que provoca un cambio de estado. Con esto se concluye que tenemos casos de uso que nos van a permitir administrar todos los estados de la clase.

Método	Caso de uso
crear()	Generar cuota comercio
registrarCedulonEmitido()	Emitir cedulón
registrarPago()	• Registrar pago en banco • Ingresar pago en caja
asignarIntimacion()	Emitir intimación a comercio
prescribir()	Procesar prescripción de cuotas

UNIDAD 7: VALIDACIÓN DE REQUERIMIENTOS

¿Qué se hace en la validación de los requerimientos?

La validación es la etapa donde se comprueba la veracidad, consistencia y completitud de los requerimientos **especificados**.

Es donde **se descubren los errores en el documento ERS y se modifica el mismo para corregir estos problemas** (cuyo costo de "reparación" va aumentando según la etapa donde se detecte, antes se detecta -> menos costo de "reparación")

"La **validación** trata de mostrar que los requerimientos especificados realmente **son los que definen al sistema que el cliente desea**"

¿Cuáles son las entradas de la validación?

- ✓ Cualquier modelo
- ✓ El conocimiento sobre el dominio del problema
- ✓ Especificación de requerimientos y prototipos de interfaz

¿Cuáles son las actividades de la validación?

- ✓ Interacción entre analistas, clientes del sistema y los usuarios donde se realiza un feedback continuo.
- ✓ Se prepara el experimento, se ejecuta y finalmente se analizan los resultados.

¿Cuáles son las salidas de la validación?

- ✓ El modelo de requerimientos que concuerde con las expectativas de los usuarios. No significa que sea el correcto, ya que ahora hay que considerar hacer un análisis técnico, económico y operativo sobre si es factible concretar la propuesta o no.

¿Cuáles son los tipos de comprobación?

De validación	De consistencia	De integridad	De realismo	Verificabilidad
Ir a lo concreto, a lo que el sistema realmente necesita y no a lo que el usuario cree que el sistema necesita.	Todos los requerimientos del sistema deben ser consistentes, no debe haber lugar a contradicciones ni descripciones diferentes de la misma función del sistema.	El documento de ERS solo debe incluir los requerimientos que definan todas las funciones y restricciones propuestas por los usuarios.	Todos los requerimientos deben ser posibles de implementar teniendo en cuenta nuestros recursos, condiciones del negocio y tecnología actual.	Los requerimientos deben estar redactados de forma que cuando se llegue a la instancia de verificación, permitan diseñar un conjunto de pruebas que demuestre que el sistema a entregar cumple con los requerimientos.

¿Cuáles son las técnicas de validación?

Revisión de requerimientos: Una vez definidos los requerimientos, son analizados sistemáticamente por un **equipo de revisores**. Es un **proceso manual** en el que varios lectores verifican el **documento de requerimientos**.

Construcción de prototipos: A partir de los requerimientos y el modelo de casos de uso, se muestra un **modelo del sistema** (versión inicial del sistema de software) a los usuarios finales y a los clientes, quienes experimentarán con el mismo para ver si cumplen con sus necesidades reales.

Existen dos tipos de prototipos:

- Prototipo **evolutivo**: a medida que se desarrolla, puede revelar errores y omisiones en los requerimientos propuestos.
- Prototipo **desechable**: se utilizan solo para la validación de los requisitos y luego se "desechan". Pueden ser en papel o en software. Sirven para eliminar dudas sobre lo que realmente quiere el cliente
 - Ejecutable
 - No ejecutable

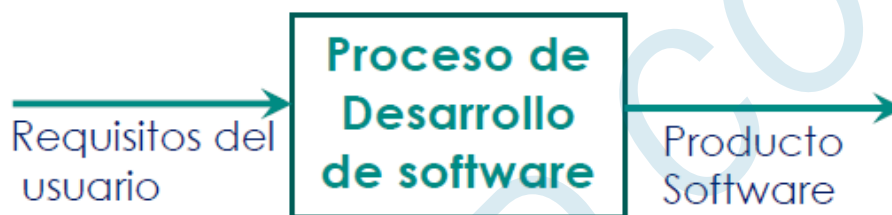
Generación de casos de prueba: Los requerimientos se tienen que poder probar. Si estas pruebas son difíciles o imposibles de diseñar, entonces significa que hay que replantear ese requerimiento.

UNIDAD 8

PUD, FLUJOS DE TRABAJO DE REQUISITOS Y ANÁLISIS

¿Qué es el Proceso Unificado de Desarrollo (PUD)?

El PUD es el conjunto de actividades necesarias para transformar los requisitos del usuario en un sistema de software.



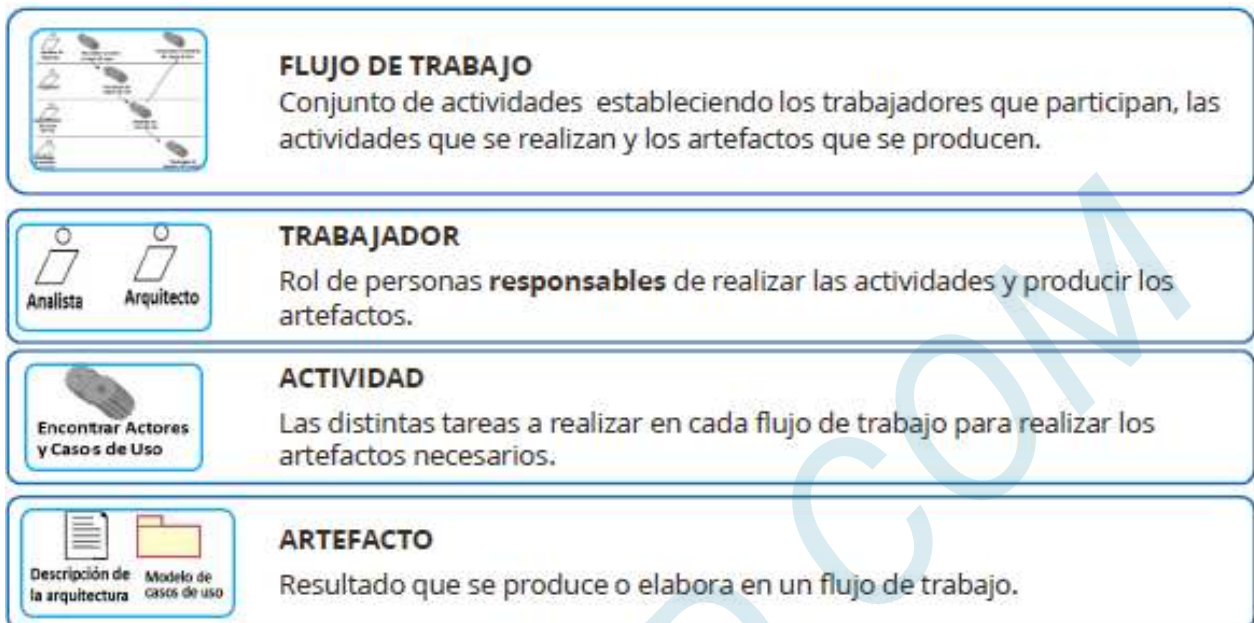
Sirve como "mapa" para desarrollar y ejecutar todas las tareas en este proceso de software.

El proceso es, capturar todos estos requisitos mediante la elicitación, aplicar diversas técnicas y métodos para construir métodos dentro de un paradigma, ejecutar las actividades, y finalmente lograr el producto que es el sistema de software.

PUD y UML

El PUD utiliza el lenguaje unificado de modelado (UML) para preparar todos los esquemas del sistema de software, sus modelos y sus vistas, siempre teniendo en cuenta lo que propone el paradigma orientado a objetos (POO).

Conceptos básicos del PUD



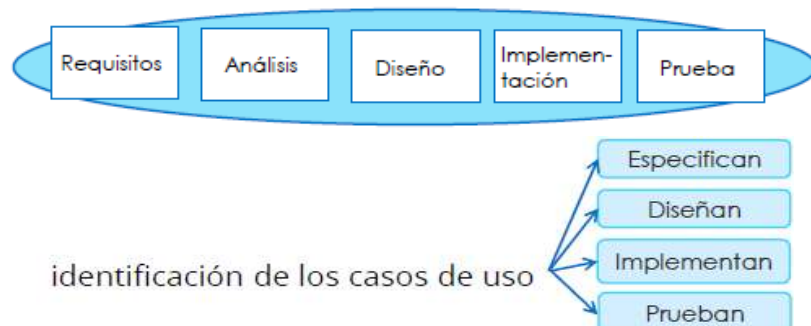
Características esenciales del PUD

1. Dirigido por casos de uso

Recordando lo que es un caso de uso: un CU es un fragmento de funcionalidad que un sistema va a ejecutar para poder cumplir con un objetivo, y que siempre va a devolver un resultado con valor para ese actor.

Son los casos de uso los que inician y guían en proceso de desarrollo.

A través de la elicitación, se capturan los requisitos funcionales y no funcionales. A partir de los funcionales se encuentran los CU. Una vez encontrados todos, el PUD va a seguir las actividades a través de una serie de flujos de trabajo, agregando especificaciones según la etapa, además de implementar y probarlos.

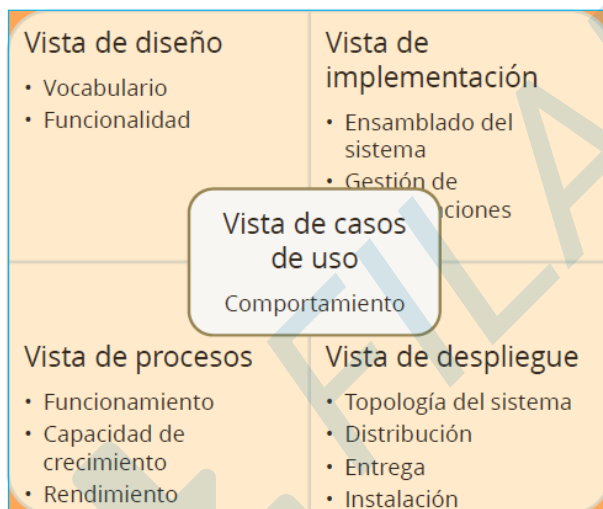


2. Centrado en arquitectura

La arquitectura del software es la que incluye a los aspectos estáticos y dinámicos más significativos del sistema.

El PUD asume que no existe un modelo único que cubra todos estos aspectos del sistema, es decir que existen varios modelos y vistas que definen la arquitectura de software de un sistema

Los casos de uso son los que tienen que “encajar” con la arquitectura cuando se llevan a cabo, a la vez que **la arquitectura es la que tiene que permitir el desarrollo** de estos casos de uso.



Vista de casos de uso	Describe el comportamiento del sistema y su funcionalidad a través de los casos de uso.	•Aspecto <u>estático</u>: Diagrama de casos de uso. •Aspecto <u>dinámico</u>: Diagramas de interacción (de comunicación y de secuencia), Diagrama de estados y de actividades.
Vista de diseño	Comprende las clases, interfaces y colaboraciones que conforman el vocabulario del problema y la solución de la funcionalidad.	•Aspecto <u>estático</u>: Diagrama de clases y diagrama de objetos. •Aspecto <u>dinámico</u>: Diagramas de interacción, Diagrama de estados y de actividades.
Vista de procesos	Comprende los hilos y procesos que forman los mecanismos de sincronización y concurrencia del sistema	Los aspectos estáticos y dinámicos de esta vista se capturan con los mismos diagramas que la vista de diseño pero poniendo énfasis en las clases activas.
Vista de Implementación	Comprende los componentes y archivos que se utilizan para ensamblar y hacer disponible el sistema físico.	•Aspecto <u>estático</u>: Diagramas de componentes. •Aspecto <u>dinámico</u>: Diagramas de interacción, Diagrama de estados y de actividades.
Vista de despliegue	Contiene los nodos que conforman la topología de hardware sobre la que se ejecuta el sistema.	•Aspecto <u>estático</u>: Diagramas de despliegue. •Aspecto <u>dinámico</u>: Diagramas de interacción, Diagrama de estados y de actividades.

3. Iterativo e incremental

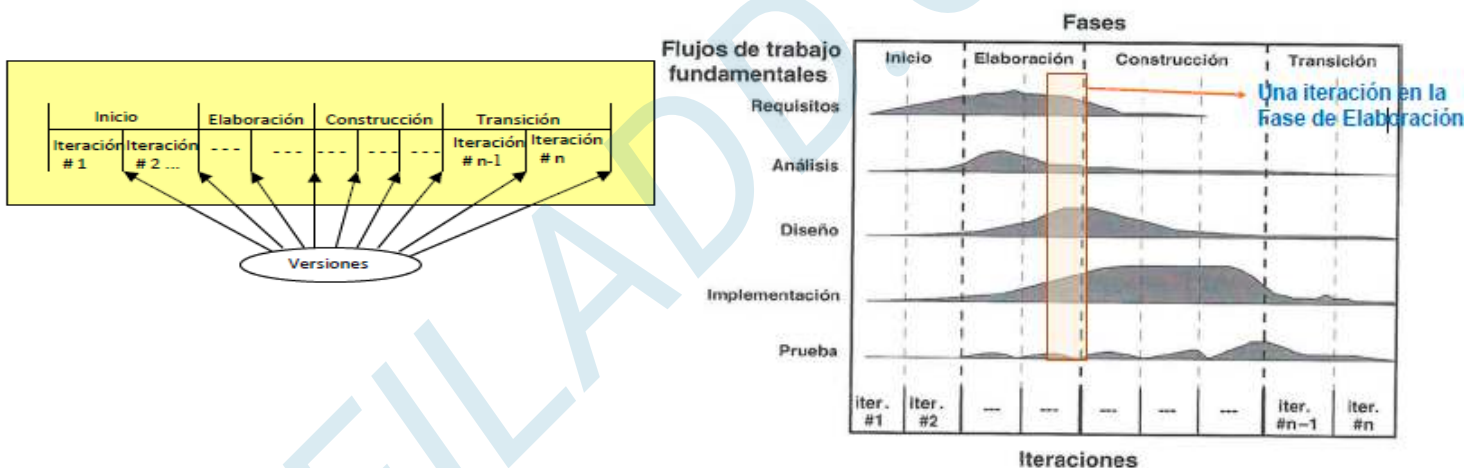
Iterativo -> hace referencia a la serie de pasos en el flujo de trabajo.

Incremental -> hace referencia al crecimiento del producto que se logra mediante cada iteración.

Cada iteración debe estar planificada y controlada en su avance para poder garantizar que los tiempos, recursos y costos que se van a usar sean bien usados.

Cada iteración produce una nueva versión del sistema para los clientes. Cada versión es un producto que ya está preparado para su entrega.

Cada fase se divide en iteraciones:



Fase Inicio: Se define el alcance del proyecto, la descripción del producto final, determinación de requerimientos funcionales y no funcionales, arquitectura del sistema, etc.

Fase Elaboración: Se especifican los casos de uso, su diagrama, descripción y diseño de prototipos.

Fase Construcción: Orientado a la codificación-programación del sistema, la creación del producto, las pruebas internas, y la tarea de verificar que el sistema funcione de acuerdo a lo solicitado.

Fase Transición: Se comienza a instalar el producto en su versión beta. Los usuarios prueban el sistema.

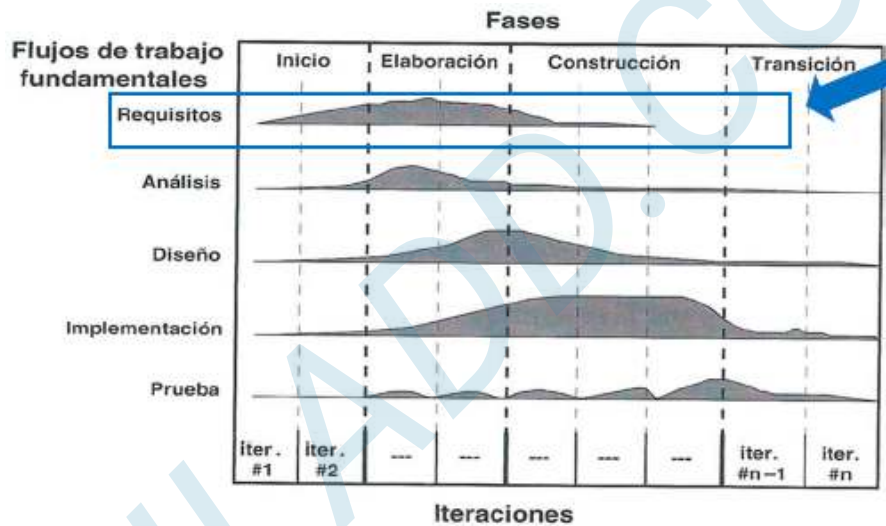
Flujo de trabajo

Flujo de trabajo de diseño: dar forma, es donde se agrega el "cómo esto va a funcionar" y construir el modelo del diseño.

Flujo de trabajo de implementación: agregar toda la parte de codificación al diseño y construir el modelo de implementación.

Flujo de trabajo de prueba: verificar que lo que está programado es acorde a los requisitos y construir el modelo de pruebas.

Flujo de Trabajo de Requisitos



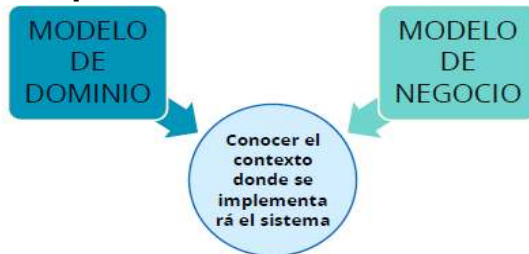
Propósitos

- ✓ Guiar el desarrollo hacia el sistema correcto.
- ✓ Hacer una descripción de los requisitos del sistema para que los desarrolladores lleguen a un acuerdo con el cliente sobre qué debe y no hacer el sistema.

Actividades previas

1. Capturar requisitos no funcionales -> generales/asociados a un CU
2. Capturar requisitos funcionales -> a través de los casos de uso

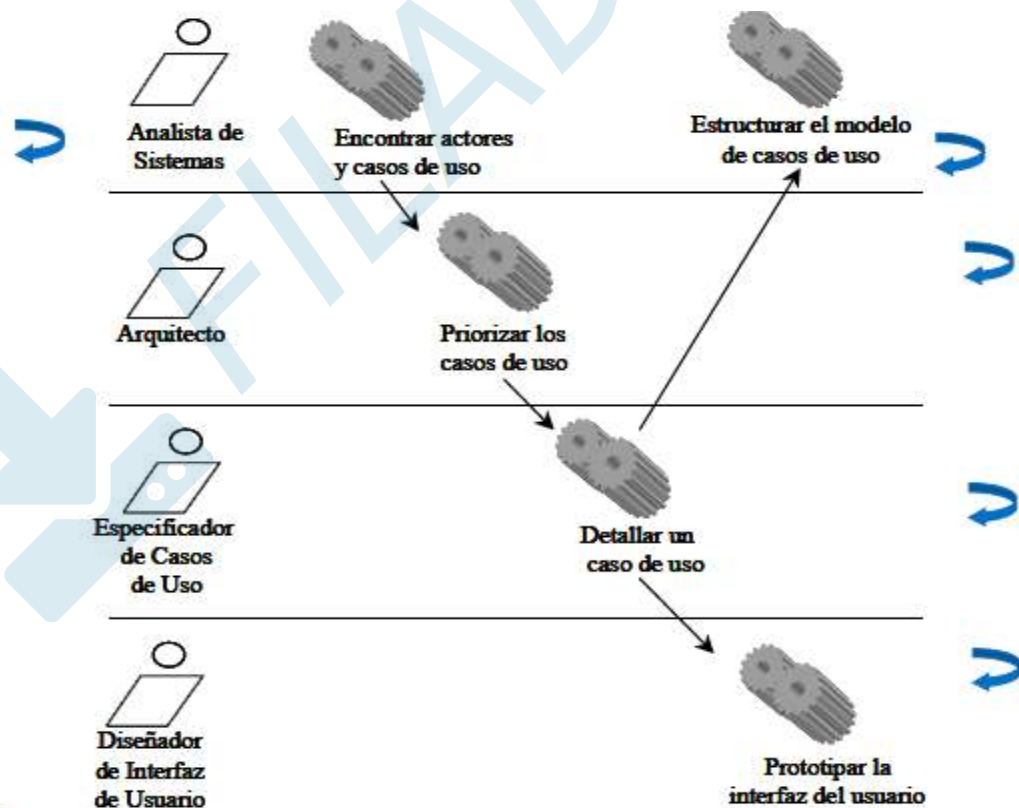
3. Comprender contexto del sistema



4. Enumerar los requisitos candidatos



Actividades



Trabajadores y Artefactos

Se distinguen cuatro roles para los trabajadores que van a participar en este flujo de trabajo:

Analistas de sistemas -> encontrar actores, casos de usos (describirlos brevemente) y modelarlos. Sus artefactos son el modelo de casos de uso (con todas las relaciones posibles entre casos de uso, incluido los patrones), el actor y el glosario.

Arquitecto -> priorizar los casos de uso. Su artefacto es la descripción de la arquitectura, la cual contiene una vista del modelo de casos de uso, que representa los casos de uso mas significativos (los que describan alguna funcionalidad importante y critica o algún requisito que deba desarrollarse pronto)

ID	Nombre del Caso de Uso	Prioridad
1	Registrar Comercio	Alta
2	Registrar Contribuyente	Alta
6	Registrar Baja de Comercio	Alta
9	Consultar Actividades	Media
19	Actualizar Cajero	Baja

Especificador de casos de uso -> detallar un caso de uso (detallar el flujo de sucesos en detalle, cómo comienza, cómo interactúa con los actores (**plantilla**) y cómo termina). Su artefacto es el caso de uso, pero en formato de plantilla.

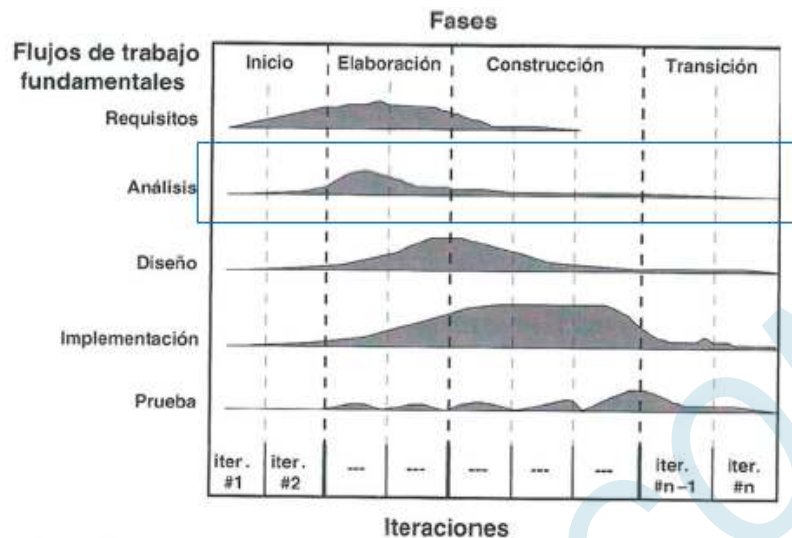
Diseñador de Interfaz de Usuario -> prototipar la interfaz de usuario. Su artefacto es el prototipo de interfaz de usuario.

Artefacto: resultado de ejecutar una actividad con el fin de lograr un producto.

Modelo de Casos de Uso: modelo que contiene actores, casos de uso y sus relaciones. Si es grande conviene introducir paquetes para agrupar casos de uso según un criterio de homogeneidad.

Glosario: se utiliza para definir términos comunes que son importantes para que los analistas y otros desarrolladores puedan utilizar para describir al sistema.

Flujo de Trabajo de Análisis



Propósitos

- ✓ **Estudiar los requisitos** que se describieron en la etapa de Requisitos, **refinándolos y estructurándolos**.
- ✓ Conseguir una comprensión más precisa de los requisitos desde una vista interna del sistema, y una descripción de los mismos, que sea fácil de mantener y que nos ayude a estructurar el sistema entero.

Aclaraciones

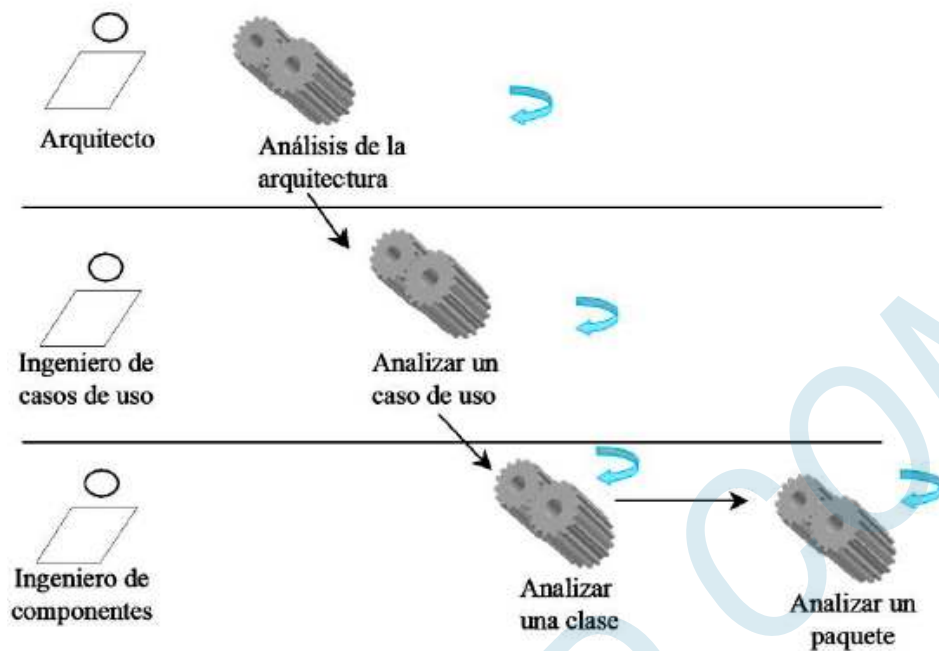
- El lenguaje que se utiliza en el análisis se basa en un **modelo de objetos conceptual** que se denomina **Modelo de Análisis**, que nos permite refinar y razonar los requisitos sobre los aspectos internos del sistema.
- La estructura del sistema que se obtiene está basada en **clases y paquetes** con una vista interna.

Modelo de casos de uso en Modelo de Requisitos

Modelo de Análisis

Descrito con el lenguaje del cliente	Descrito en el lenguaje del desarrollador
Vista externa del sistema	Vista interna del sistema
Estructurado por casos de uso	Estructurado por clases y paquetes
Utilizado fundamentalmente como contrato entre el cliente y equipo de sistema sobre qué debería hacer el sistema y qué no	Utilizado fundamentalmente por los ingenieros en sistemas para comprender cómo debería darse forma al sistema
Captura la funcionalidad del sistema	Esboza cómo llevar a cabo la funcionalidad dentro del sistema; primera aproximación al diseño
Define casos de uso que se analizarán con más profundidad en el modelo de análisis	Define realizaciones de casos de uso y cada una de ellas representa el análisis de un caso de uso

Actividades



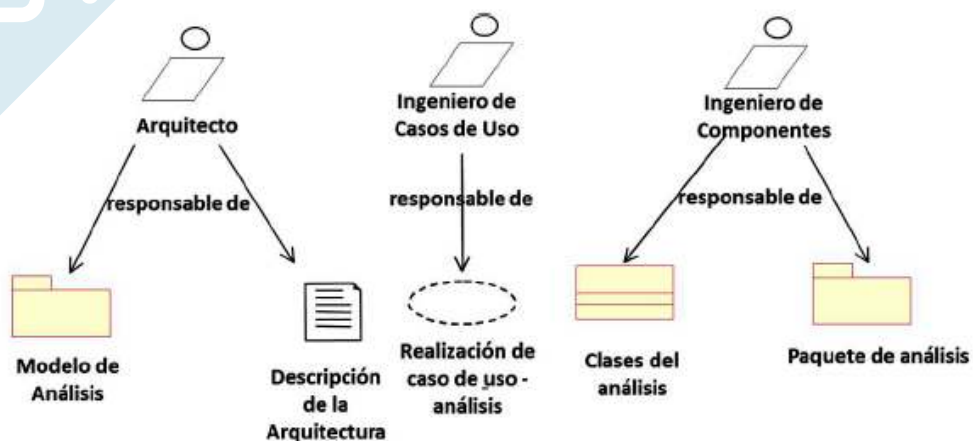
Análisis de la arquitectura: se **identifican paquetes de análisis, clases de entidad y requisitos especiales comunes** -> persistencia, distribución, concurrencia, seguridad, tolerancia a fallos, etc.

Analizar un caso de uso: **identificar clases de análisis* participantes** mediante el diagrama de clases de análisis y **describir interacciones entre objetos de análisis** mediante los diagramas de interacción.

Analizar una clase: Identificar responsabilidades, atributos, relaciones.

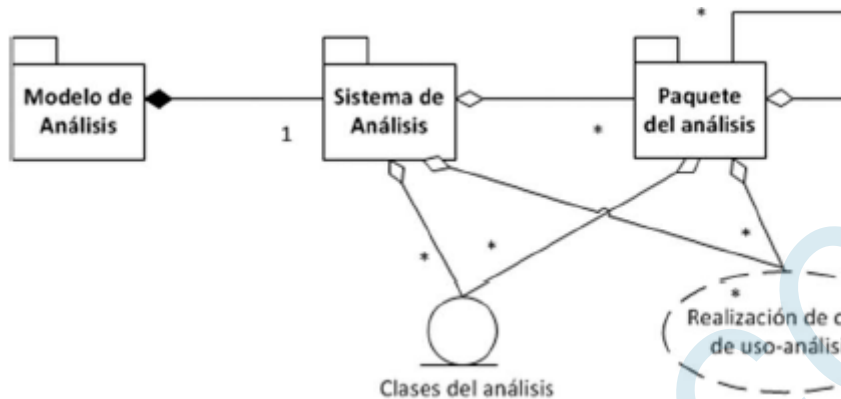
Analizar un paquete: Garantizar que tenga **poco acoplamiento**, que **cumple su objetivo**, y que tenga **alta cohesión**.

Trabajadores y artefactos



Arquitecto

Modelo de análisis: está organizado en paquetes de análisis que son abstracciones de subsistemas. Los Casos de uso se describen mediante **clases de análisis** y sus **objetos**, y se representan mediante colaboraciones.



Descripción de la arquitectura: contiene una vista del modelo de análisis que **muestra los artefactos mas significativos para la arquitectura.**

Ingeniero de Casos de Uso

Realización de caso de uso-análisis: **describe cómo se lleva a cabo y se ejecuta un caso de uso, teniendo en cuenta la interacción entre objetos de análisis.** Ejemplo: si tenemos un caso de uso, que en uno de sus pasos dice "El sistema muestra los datos del cliente", esa seria una vista externa, entonces el ingeniero va a adoptar una mirada interna para ver cómo es que el sistema hace para mostrar esos datos del cliente, observando los objetos que intervienen y colaboran entre sí, si un objeto tiene información..., etc.

Ingeniero de componentes

Clase de análisis: es una **abstracción que modela un elemento específico del dominio del problema**, centrándose en el **tratamiento de los requisitos funcionales**. Su comportamiento se define mediante responsabilidades. Tiene atributos de un alto nivel de abstracción. También participa en relaciones. Encaja en uno de los tres estereotipos básicos:

Estereotipo	Icono	Significado (semántica)
<<boundary>>		Una clase mediadora entre el sistema y su entorno (actores)
<<control>>		Una clase que encapsula comportamiento específico de un caso de uso
<<entity>>		Una clase que se utiliza para modelar información persistente sobre algo

Análisis de clases*:

Clase	Se identifican . . .
Entidad 	1. Mediante el estudio detallado de la descripción del caso de uso y del modelo de dominio. 2. Considerar qué información debe utilizarse en el caso de uso. 3. Separar información que se puede capturar en un atributo, o que se puede asociar a una clase de interfaz o control.
Interfaz 	1. Definiendo inicialmente una por cada actor humano. 2. Definiendo una clase de interfaz central para cada actor que sea un sistema experto, a fin de que represente la interfaz de comunicación.
Control 	1. Definiendo una clase de control responsable del tratamiento de control y de la coordinación de la realización del caso de uso. 2. Luego refinar esta clase de control de acuerdo a los requisitos del caso de uso, considerando que puede haber casos de uso muy complejos en los que es mejor distribuir el control entre dos o más clases de control.

Paquete de análisis: Es un mecanismo lógico de agrupación que sirve para organizar elementos de un modelo. Organiza los artefactos del modelo de análisis en piezas manejables. Un paquete puede contener clases de análisis, realizaciones de casos de uso e incluso otros paquetes de análisis.